Training a VLA on SO-101

A Vision-Language-Action model (VLA) looks through the arm's cameras, reads one sentence from you, and answers with the next 1.7 seconds of joint motion. This lesson explains how that works, what you must prepare (hardware, software, data), how to train and run it on the SO-101, and what goes wrong at every step.

Level: beginner | Builds on: imitation learning with ACT | Hardware: SO-101 leader + follower, 2 cameras | Model used: SmolVLA (450M)

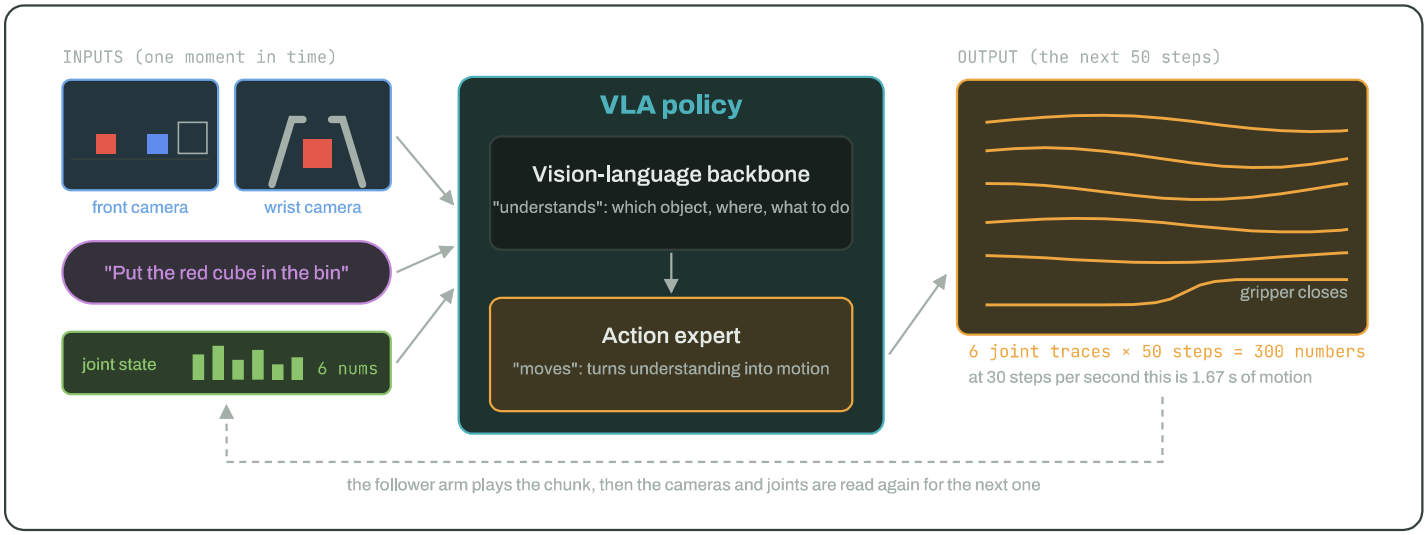


Figure 1. The whole idea on one page. Colours are used the same way in every figure of this lesson: **blue = vision**, **plum = language**, **green = joint state**, **amber = action**.

What you will be able to do

EXPLAIN

How a VLA turns pixels and a sentence into joint motion, and how it differs from ACT.

PREPARE

An SO-101 rig, a working LeRobot install and a dataset designed so that the instruction actually matters.

TRAIN & RUN

Fine-tune SmolVLA on your episodes and run it on the real arm without stutter.

JUDGE

Measure success honestly, test whether the robot listens, and trace each failure back to the stage that caused it.

BEFORE YOU START

You should already have teleoperated the SO-101 (leader moves, follower copies) and trained an ACT policy in the imitation-learning lab. This lesson reuses those ideas: **episodes**, **action chunks**, the **action queue** and **demonstration quality**. If a word is new, check the glossary.

Contents

1. From ACT to VLA
2. Inside a VLA & how it is built
3. The maths, simplified
4. How your data trains it
5. Stage 1 · Prepare the rig
6. Stage 2 · Install the software
7. Stage 3 · Collect the data
8. Stage 4 · Train
9. Stage 5 · Deploy
10. Stage 6 · Evaluate
11. Lab plan
12. Troubleshooting
13. Glossary

PART A1 CONCEPT

From ACT to VLA: what changes

So far in this unit you have met two ways to give the SO-101 a skill. In **reinforcement learning** the arm tries things in simulation and a reward score tells it how well it did. In **imitation learning** (ACT) you show it the task with the leader arm and it learns to copy you. A VLA is a third step on the same ladder: it is still imitation learning, but the network starts from a model that has already learned to see and read from a very large amount of data.

REINFORCEMENT LEARNING

Learns by trial and reward

YOU GIVE IT

A reward function and a simulator

IT ALREADY KNOWS

Nothing

GOOD FOR

Reaching, balance, skills you can score with a formula

IMITATION LEARNING · ACT

Learns by copying you

YOU GIVE IT

~50 demonstrations of one task

IT ALREADY KNOWS

Only generic image features (ImageNet backbone)

GOOD FOR

One fixed task, done precisely

VISION-LANGUAGE-ACTION

Copies you, but starts educated

YOU GIVE IT

Demonstrations *plus* a sentence for each

IT ALREADY KNOWS

Objects, colours, words, and motion from thousands of other robot episodes

GOOD FOR

Several tasks chosen by instruction; more tolerance to new scenes

ANALOGY

Training ACT is like hiring someone who has never seen a kitchen and showing them how to make one sandwich fifty times. Training a VLA is like hiring a trained cook who already knows what bread, knives and "toasted" mean. You still have to show the cook *your* kitchen and *your* sandwich, but they pick it up faster and can follow "make the cheese one this time". Part A2 shows how this cook is trained.

The models you can use with SO-101 today

All of these run inside Hugging Face **LeRobot**, the open-source library the SO-101 was designed for. This lesson uses SmolVLA because it is small enough for a lab GPU and was pre-trained largely on community SO-100/SO-101 data.

MODEL	TAKES AN INSTRUCTION?	SIZE	PRE-TRAINED ON	TRAINING GPU (ROUGH)	USE IT
ACT	No	tens of M	ImageNet image features only	8 GB is enough	Baselin One tas one sce
SmolVLA	Yes	450	481	16–24 GB	First V

		M	community LeRobot datasets (~23k episodes, 10.6 M frames), many on SO-100 arms	comfortable; 8 GB only with a small batch; ~4 h for 20k steps on one A100	on SO- This lesson.
π0 / π0.5 (Physical Intelligence)	Yes	~3.3 B	Large multi-robot dataset	Full fine-tune needs an 80 GB-class GPU; LoRA reduces this	Harder contact rich tas when y have cl GPUs
GR00T N1.x (NVIDIA)	Yes	~3 B	Real, simulated and video data across embodiments	N1.5 default fine-tune ≈ 25 GB	Links t the Isa Sim / Is Lab sid the cou

The ACT figure matches the earlier ACT lesson (about 34 M parameters used at inference). LeRobot v0.6 replaced GR00T N1.5 with N1.7; check the docs for the exact version your install ships with.

PART A2 CONCEPT

Inside a VLA, and how the "trained cook" is built

In Part A1 we said a VLA is like hiring a trained cook: someone who already knows what bread, knives and "toasted" mean, so you only have to show them your kitchen and your sandwich. This part opens that analogy up. First the parts of the model, then what each of the three letters V, L and A adds, then the five stages of training that turn an empty network into a cook you can hire, and finally how you teach that cook your task, and later a new one.

A2.1 · The five parts: eyes, ears, body sense, brain, hands

Every VLA in the table above has the same five parts. The figure uses SmolVLA's real numbers.

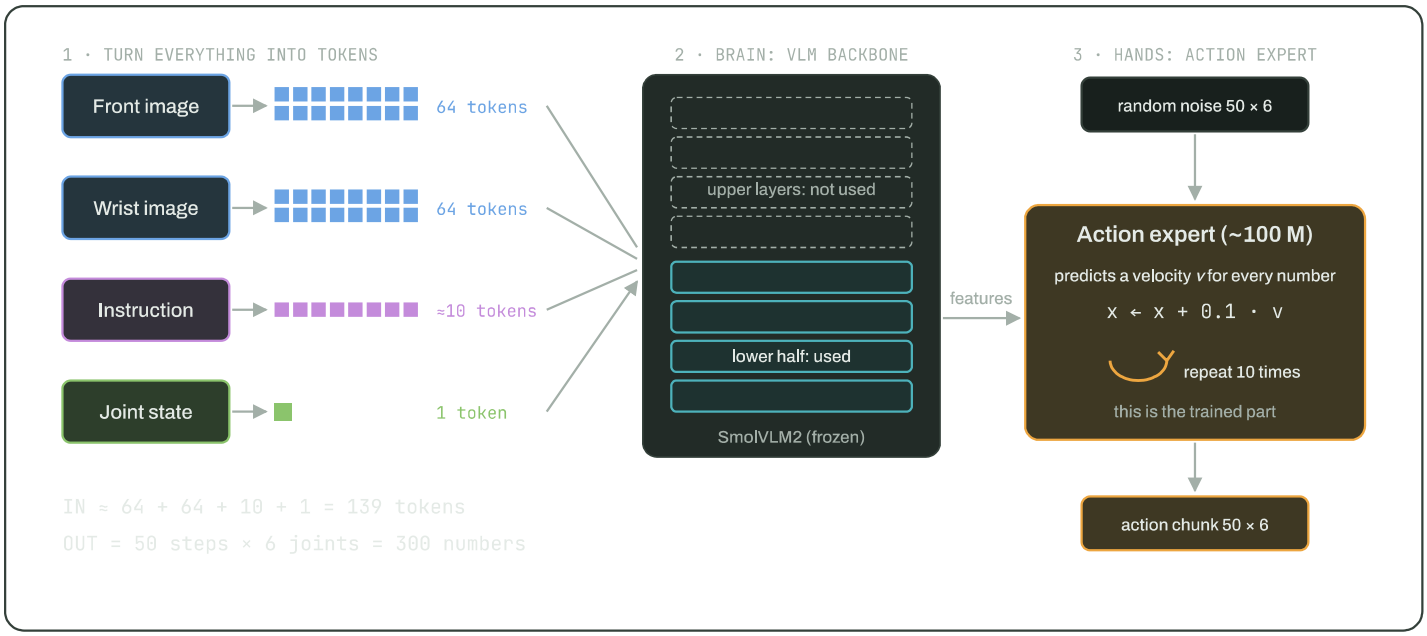


Figure 2. SmolVLA, part by part. The token count for the sentence depends on the wording; 10 is a typical value for a short instruction. Only the coloured action expert (and a few small connector layers) is updated during fine-tuning by default.

PART	BODY ANALOGY	WHAT IT DOES IN PLAIN WORDS	SMOLVLA DETAIL
Vision encoder	Eyes	Cuts each camera image into small patches and turns each patch into a list of numbers (a *token*) that describes what is there.	Each frame is squeezed to 64 tokens so the model stays fast.
Tokenizer	Ears	Splits your sentence into word pieces and gives each one a token.	Same tokenizer as the language model.
State projector	Body sense	Turns the 6 joint readings into one token, so the model knows where the arm is now.	One linear layer.
VLM backbone	Brain ("System 2", slow thinking)	Reads all tokens together and relates them: "red cube" ↔ the red pixels in the front image ↔ the gripper's position.	SmolVLM2, kept frozen. Only the lower half of its layers is used, which halves the cost.
Action expert	Hands ("System 1", fast reflex)	A smaller network that turns the brain's understanding into a smooth 50-step motion plan.	About 100 M parameters, trained with flow matching (Part A.2)

(Part A3).

A2.2 · What V, L and A each add

The easiest way to see what each part contributes is to take it away. Figure 2b shows the same SO-101 task with one letter missing each time.

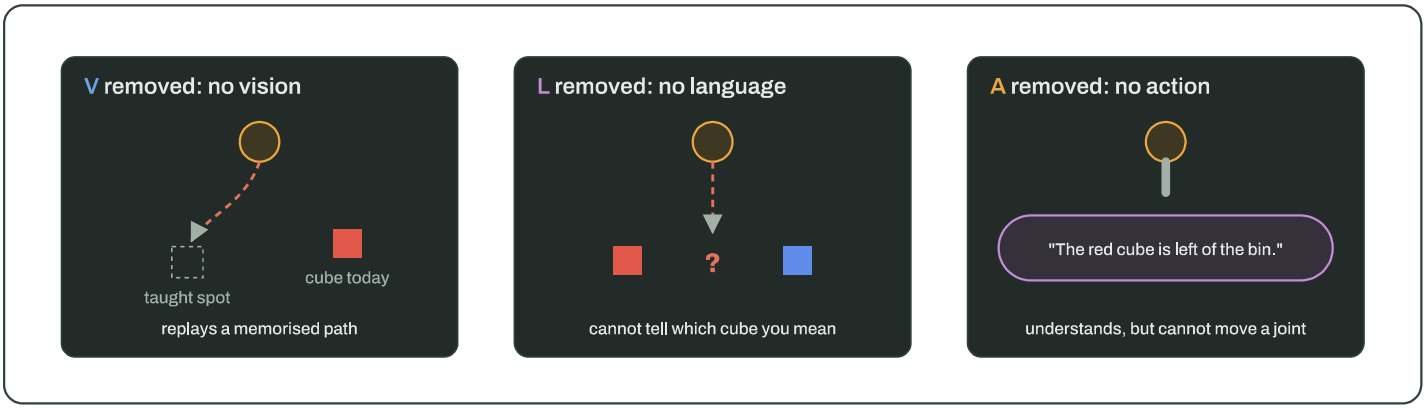


Figure 2b. Take one letter away. Without V you get a tape recorder, without L you get a one-task robot, and without A you get a chatbot that can see.

LETTER	ABILITY IT ADDS	SO-101 EXAMPLE	COOK ANALOGY
V · Vision	Finds and recognises objects in the camera images, so the motion adapts to where things are	The cube is 4 cm further left than in the demos; the arm reaches there instead of to the old spot	Looks at the counter before reaching for the bread
L · Language	Chooses *which* task or object, and links words to things it sees	"red" vs "blue" picks the cube; one model holds several tasks	Understands the order: "the cheese one this time"
A · Action	Turns understanding into joint motion for this particular arm	A 50 × 6 chunk of joint targets the follower can play	Hands that can actually slice and spread
V + L + A	Instruction following, task switching, tolerance to small scene changes, and reuse of general knowledge	"Put the blue cube in the bin" with the cubes in new places	A cook who reads the order, looks at the counter and makes it

Compare this with what you already know:

MODEL	V	L	A	RESULT
A chat model that sees images (VLM)	✓	✓	–	Describes the scene in words; cannot drive motors
ACT	✓	–	✓	Moves well, but one task per model; vision learned only from your 50 demos
VLA (SmolVLA)	✓	✓	✓	Moves, chooses the task by instruction, and brings pre-trained knowledge of what things look like and are called

A2.3 · How the cook was trained: five stages

Nobody trains a VLA in one go. It is assembled from models that were each trained for something simpler, and every stage reuses what the previous stages learned. Stages 1–4 were done for you by Google, Hugging Face and the LeRobot community; stage 5 is your lab.

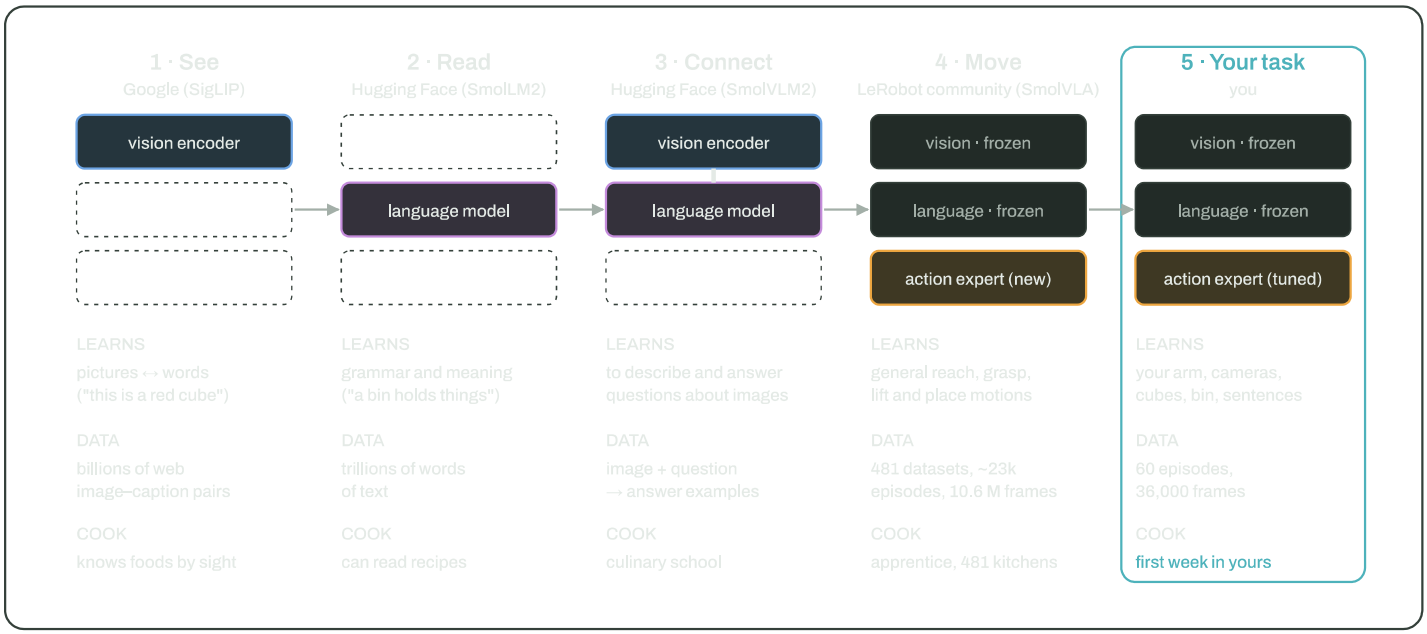


Figure 2c. The cook's education. A coloured block is trained in that stage; a grey block is reused but frozen; a dashed outline means the part does not exist yet. The heavy line in stage 3 is the small connector that joins the vision encoder to the language model.

Stage 1 · Learning to see (the vision encoder)

SmolVLA's eyes are a **SigLIP** vision encoder, trained by Google on a very large collection of web images and their captions. It never learns robotics. It learns one thing: turn an image into a vector of numbers that lands close to the vector of its

thing: turn an image into a vector of numbers that lands close to the vector of its matching caption, and far from wrong captions. Each image–caption pair gets a match score s, which is turned into a probability:

DOES THIS CAPTION BELONG TO THIS IMAGE?

$$p = \frac{1}{1 + e^{-s}} \qquad \text{loss} = -\log p \ \text{(true pair)} \ \text{ or } \ -\log(1 - p) \ \text{(wrong pair)}$$

WORKED EXAMPLE	SCORE S	P	LOSS	TRAINING DOES…
photo of a red cube + "a red cube" (true)	3.0	0.95	0.05	almost nothing; already right
photo of a red cube + "a blue cube" (wrong)	2.0	0.88	2.13	pushes this score down hard

Repeat this over billions of pairs and colours, shapes, objects and positions all get their own regions in the vector space. That is why the 64 image tokens SmolVLA receives are already meaningful before it has ever seen your bench.

Stage 2 · Learning to read (the language model)

The brain's language half is **SmolLM2**, a small language model trained by Hugging Face on trillions of words of text. Its only task: predict the next word.

NEXT-WORD LOSS

$$\text{loss} = -\log p(\ \text{correct next word} \mid \text{the words so far}\ )$$

"PUT THE CUBE IN THE ___"	MODEL'S P("BIN")	LOSS
after a lot of training	0.60	0.51
early in training	0.05	3.00

To predict well, the model has to pick up grammar and a lot of everyday meaning: that bins and boxes hold things, that "put X in Y" means X ends up inside Y, that "red" and "blue" are both colours. None of this is about robots, and all of it is useful to one.

Stage 3 · Connecting seeing and words (the vision-language model)

Hugging Face then joined the two into **SmolVLM2.** A small connector squeezes the encoder's image features into 64 tokens and feeds them into the language model as if they were words. The joined model is trained on examples of the form *image + question → answer*: captions, "how many cubes are there?", reading text in pictures, short videos. Now it can look at your bench photo and answer "The red cube is to the left of the bin."

This is the cook leaving culinary school: they can read a recipe while looking at the ingredients. But the output is still **words**. Nothing in stages 1–3 knows what a servo is.

Stage 4 · Learning to move (SmolVLA pre-training)

The LeRobot team added the **action expert**, a new network of about 100 M parameters that starts with random weights, and trained it on robot data shared by the community on the Hugging Face Hub:

Data	481 community datasets, about 23,000 episodes and 10.6 million frames, mostly low-cost SO-100-type arms in many different homes and labs
Cleaning the language	Community task labels were often vague ("task1", "pick"). A separate VLM (Qwen2.5-VL-3B) looked at sample frames and rewrote each label as a short, action-style instruction
Cleaning the cameras	Camera names differed in every dataset, so views were mapped to fixed slots: top, wrist, side → image 1, 2, 3. This is why your own fine-tune may need `--rename_map` to the base model's camera names (Part B, Stage 4: Train)
What is trained	The action expert, with the flow-matching loss you will meet in Part A3. The vision-language model stays frozen, so its knowledge from stages 1–3 is kept
Result	`lerobot/smolvla_base` : a model that already produces sensible reach-grasp-place motion for arms like yours, steered by images and an instruction

This is the apprenticeship. The cook has worked in 481 kitchens, each with different counters, lighting and utensils, so they have general hand skills that do not depend on any one kitchen.

Stage 5 · Your kitchen (fine-tuning)

You start from `smolvla_base` and continue training on your own 60 episodes. The same flow-matching loss is used; the backbone stays frozen and the action expert is

adjusted. After about 20,000 steps the model has learned the things only your bench can teach: your exact cameras and their angles, your calibration, the size of your cubes, where the bin is, and your two instructions.

STAGE	PART TRAINED	DATA	LOSS	ABILITY GAINED
1 · See	Vision encoder	Image–caption pairs	Image–text match	Recognise objects and colours
2 · Read	Language model	Text	Next word	Understand instructions
3 · Connect	Both + connector	Image + question → answer	Next word	Talk about what it sees
4 · Move	Action expert (new)	Community robot episodes	Flow matching	General arm motion from images + words
5 · Your task	Action expert	Your 60 episodes	Flow matching	Your arm, scene and instructions

A2.4 · Why 60 episodes are enough: transfer

Compare the experience behind each stage:

COMMUNITY ROBOT DATA VS YOUR DATA

$$\frac{10.6\text{ M frames}}{36{,}000\text{ frames}} \approx 294\times$$ at 30 fps: ≈ 98 hours of demos vs 20 minutes

Your 20 minutes could never teach grasping from scratch on top of what a red cube looks like and what "bin" means. They do not have to. Training always changes weights a little at a time:

ONE TRAINING UPDATE (ONLY FOR WEIGHTS THAT ARE NOT FROZEN)

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

where η is a small step size. Where you *start* decides how far you have to walk. Figure 2d shows the idea on a one-dimensional loss curve.

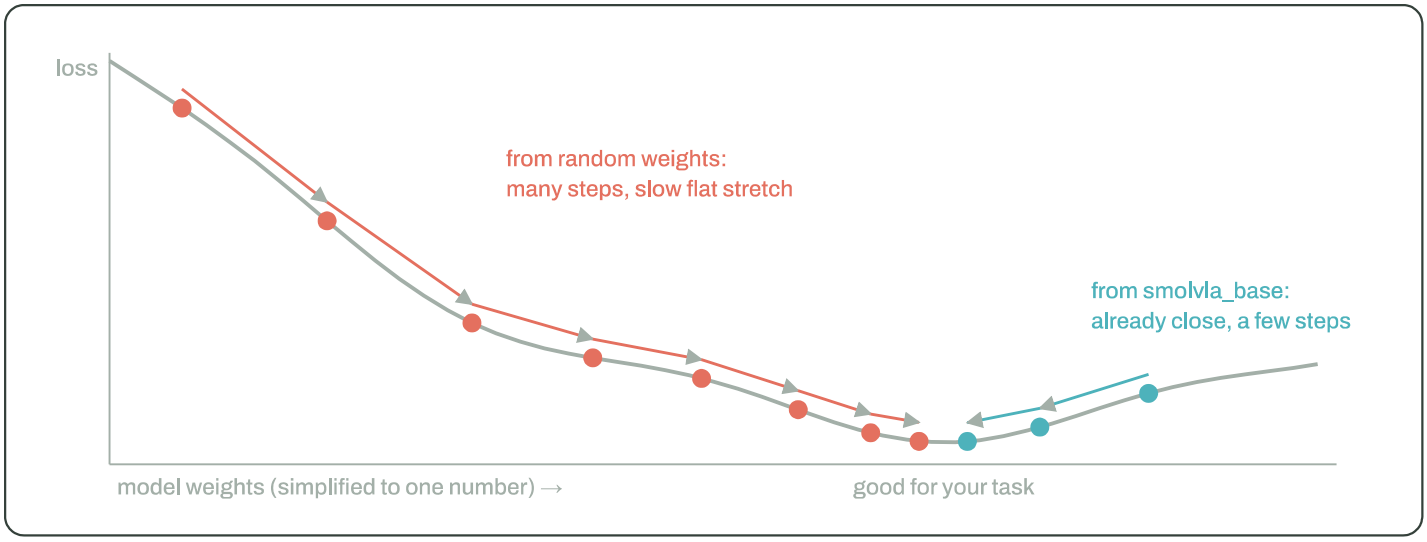


Figure 2d. Pre-training gives a better starting point. Real networks have hundreds of millions of weights instead of one, but the picture is the same: the pre-trained model starts near a good solution, so a small dataset is enough to finish the job.

The SmolVLA authors measured this directly. In their ablation, adding community pre-training raised the average success rate from **51.7% to 78.3%** with the same fine-tuning. Their documentation also reports that about 50 episodes worked while a 25-episode version of the same task performed poorly: pre-training lowers the amount of data you need, it does not remove it.

WORKED EXAMPLE · SMOLVLA DURING YOUR FINE-TUNE	VALUE
Total parameters	≈ 450 M
Trained (action expert and small connectors)	≈ 100 M
Frozen (vision + language, stages 1–3)	≈ 350 M
Share of the model you change	≈ 22%

Freezing the backbone does two jobs: it keeps the cook's general knowledge from being overwritten by your tiny dataset, and it makes training cheap enough for a lab GPU.

A2.5 · What the cook can and cannot do after your fine-tune

The analogy says the cook "can follow *make the cheese one this time*". That is true when

the cook has made cheese sandwiches before and you showed them how sandwiches are made in your kitchen. It is a promise about **choosing between practised skills and familiar objects**, not about learning new skills from words alone.

INSTRUCTION AT TEST TIME	IN YOUR DEMOS?	LIKELY RESULT	WHY
"Put the red cube in the bin", cube at a taught spot	Yes	works	Exactly what was practised
Same, cube between two taught spots	Nearby	usually works	Vision interpolates positions
"Put the blue cube in the bin"	Yes, balanced with red	works	Language chooses between practised options
"Put the green cube in the bin" (never shown)	No, but same skill	sometimes	Stage 1–3 knowledge of "green" may carry over; test it, do not assume it
"Drop the red block into the container" (new wording)	Same task, new words	sometimes	The language model knows the words are similar, but your fine-tune only saw one phrasing
"Stack the red cube on the blue cube"	No, new skill	fails	Understanding the sentence does not give the hands a motion they never learned on this arm

A2.6 · Teaching the cook a new task later

Suppose after the lab you want the SO-101 to also handle a green cube, or to stack. You do not start over; you run stage 5 again with extra data. The question is what to start from and what data to include.

1. **Decide what kind of "new" it is.** A new *object* with the same skill (green cube into the bin) needs few episodes. A new *skill* (stacking) needs a full set, about 50 or more.
2. **Record the new episodes** on the same rig, with the same cameras and calibration, and one exact instruction string for the new task, e.g. `Put the green cube in the bin`. Keep the paired-scene rule: the green cube sits among the other cubes, so the

instruction matters.

3. **Mix old and new data.** If you train only on the new task, nothing in the data reminds the model of the old ones and it gradually forgets them (called *catastrophic forgetting*). Train on the old episodes plus the new ones together.
4. **Choose the starting point** (table below), fine-tune, and keep several checkpoints.
5. **Evaluate the old tasks again**, not only the new one. A new skill that costs you the old ones is not progress.

START FROM…	GOOD WHEN	WATCH OUT FOR
Your previous fine-tuned checkpoint	Same rig, new object or a close variation; quickest	Forgetting if the old data is left out
`smolvla_base` **again**	A new skill, a changed camera layout, or the old model has drifted	Takes the full training time; needs all the data at once

WORKED EXAMPLE · ADDING THE GREEN CUBE	EPISODES	SHARE OF TRAINING BATCHES
Old: red (30) + blue (30)	60	67%
New: green, 6 per position × 5	30	33%
Combined dataset	90	100%

Batches are sampled at random from all frames, so each task's share of training is roughly its share of the data. If the new task is lagging after evaluation, record more of it rather than dropping the old episodes.

KEY IDEA

Stages 1–3 give the cook eyes and a vocabulary, stage 4 gives general hands, and stage 5, your lab, teaches this arm in this kitchen. Your dataset only has to cover what is specific to your bench, which is why 60 episodes can be enough, and why the instruction only works for tasks your data actually shows.

PART A3 CONCEPT · SIMPLIFIED MATHS

The maths you need, and nothing more

Four small ideas explain almost everything the training is doing. Each one comes with a worked example you can check by hand.

Idea 1 · An action is a list of six numbers; a chunk is a table of them

At any moment the follower arm is told where each of its six motors should go. That command is the **action**:

ONE ACTION (TIME STEP T)

$$a_t = [\ \text{pan, lift, elbow, wrist_flex, wrist_roll, gripper}\ ]$$

A VLA does not predict one action at a time. It predicts a **chunk** of the next H actions in one go, like a short motion plan:

ACTION CHUNK

$$A_t = [\ a_t, a_{t+1}, \dots, a_{t+H-1}\ ] \ \rightarrow\ \text{size } H \times 6$$

WORKED EXAMPLE	VALUE	HOW
Chunk length H (SmolVLA default)	50	given
Numbers predicted per chunk	300	50×6
Control rate	30 Hz	same as camera fps
Time between steps, Δt	33.3 ms	$1 / 30$ s
Time covered by one chunk	1.67 s	50×33.3 ms

Why chunks? One action at a time makes small errors pile up step after step, and the motion jitters. A chunk is planned as one smooth piece, the same reason ACT uses chunks.

Idea 2 · Normalisation puts every joint on the same ruler

The shoulder may move across 100 units while the wrist roll moves across 20. If we trained on raw numbers, the network would care mostly about the big joints. So every value is rescaled using the average (μ) and spread (σ) measured over your whole dataset:

NORMALISE (TRAINING)

$$z = \frac{a - \mu}{\sigma}$$

AND BACK (DEPLOYMENT) $a = z \cdot \sigma + \mu$

JOINT (ILLUSTRATIVE NUMBERS)	μ	σ	RAW a	z
shoulder lift	20	15	50	(50 - 20) / 15 = 2.0
gripper	30	20	10	(10 - 30) / 20 = -1.0

After normalisation both joints live roughly between −3 and +3. LeRobot computes μ and σ automatically and stores them with the dataset (`meta/stats.json`) and with the trained model. Some models (π0.5) use percentiles instead of μ and σ so that one wild value cannot stretch the scale.

WHY THIS MATTERS FOR YOU

The statistics describe *your* calibration. If you recalibrate the arm after recording, the same physical pose gives different raw numbers, and every prediction is shifted. Calibrate once, then record, train and deploy with the same calibration file.

Idea 3 · Flow matching: start from noise, walk to an action

The action expert does not output the chunk directly. It starts from a table of random numbers and repeatedly nudges it towards a sensible motion. Think of a ball rolling downhill into the nearest valley; each valley is one good way to do the task.

Why not just predict the answer directly? Because good demonstrations often disagree. Look at Figure 3.



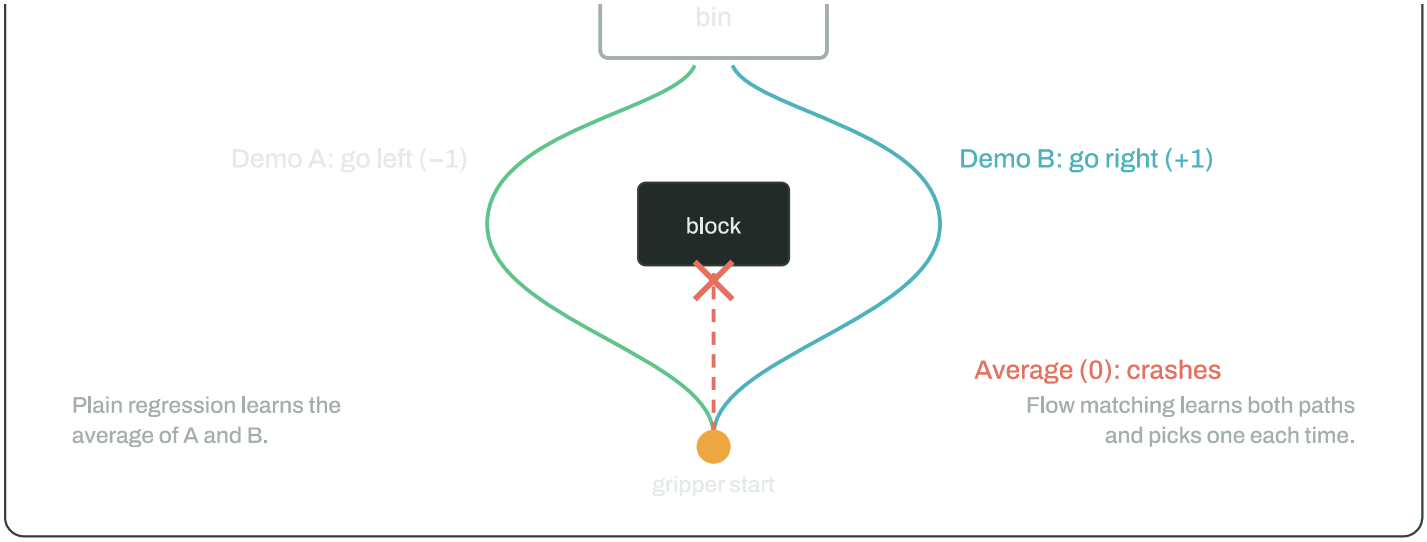


Figure 3. The averaging trap. Half your demos went left of the block, half went right. A network trained to minimise squared error learns the mean, which is a path no human ever took. A generative method such as flow matching keeps both answers and commits to one of them per chunk.

The training recipe, in three lines. Take a real chunk A from your data and a random chunk ε of the same size. Pick a random mixing amount τ between 0 and 1:

1 · MIX NOISE AND DATA

$$x_\tau = (1 - \tau) \cdot \varepsilon + \tau \cdot A$$

2 · THE DIRECTION FROM NOISE TO DATA (THE "CORRECT VELOCITY")

$$u = A - \varepsilon$$

3 · LOSS: HOW WRONG WAS THE PREDICTED VELOCITY V?

$$L = \| v_\theta(x_\tau, \tau \mid \text{images, words, state}) - u \|^2$$

At deployment the model starts from pure noise ($\tau = 0$) and takes 10 steps of size 0.1, each time moving in the direction it predicts: `x ← x + 0.1 · v`. After 10 steps τ has reached 1 and x is an action chunk.

TRY IT **One number, ten steps**

Follow a single number of the chunk, the gripper value. Its demonstrated target is $A =$

0.8 (normalised). Choose where the noise starts and drag τ.

NOISE START ε

-1.2

PROGRESS τ

0.3



-0.60

$x_\tau = (1 - \tau)\cdot\varepsilon + \tau\cdot A$

2.00

correct velocity u = A − ε (the same at every step)

0.20

distance moved per step = 0.1 × u

After 3 of 10 steps the value has moved from −1.20 to −0.60. If the network predicts u perfectly, it lands exactly on 0.80 at τ = 1.

STEP	τ	x (ε = -1.2, A = 0.8)
start (pure noise)	0.0	-1.20
after step 1	0.1	-1.00
after step 3	0.3	-0.60
after step 5	0.5	-0.20
after step 10 (finished)	1.0	0.80

In the real model the velocity is not known; the network predicts it from the images, words and state, for all 300 numbers at once. Different noise starts can end in different valleys, which is how one model can hold both "go left" and "go right".

Idea 4 · Attention: how a word picks an object

Inside the backbone, each token asks every other token "how relevant are you to me?". The answer is a score; the scores are turned into weights that add up to 1 using **softmax**:

ATTENTION WEIGHTS

e^{s_i}

$$w_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Worked example: the word token "red" scores three image patches.

IMAGE PATCH	SCORE s	e^s	WEIGHT w
red cube	2.0	7.39	0.73
blue cube	0.5	1.65	0.16
bin	0.1	1.11	0.11
sum		10.14	1.00

73% of what "red" passes on comes from the red cube's patch, so the action expert receives features centred on the red cube. Change the word to "blue" and the weights shift to the blue cube. **This only works if, during fine-tuning, the word actually decided which cube was correct.** Stage 3 shows how to make sure of that.

PART A4 CONCEPT · DATA TO MODEL

Same recordings, different learner: how your data trains a VLA

For ACT you recorded the front and wrist (gripper) camera images, the joint states and the leader-arm actions. For SmolVLA you record the same things. Figure 1 shows a vision-language backbone *and* an action expert, so it is fair to ask: how can the same small dataset train a much bigger model with more parts? The answer has three parts:

1. **There is one extra field.** Every episode also stores its instruction text, the `task`. LeRobot records it for ACT too (it is the `--dataset.single_task` you typed), but ACT ignores it. SmolVLA reads it.

2. **Your data does not train the vision-language backbone.** That part was trained on web images and text long before (Part A2, stages 1–3) and stays frozen. Your data trains only the action expert and the small state projector.

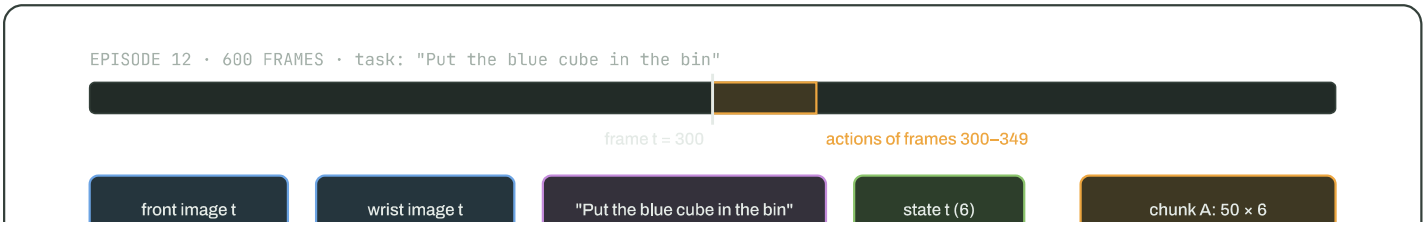
3 **The teaching signal is the same as in ACT.** A VLA fine-tune is still imitation learning: the recorded actions are the only "answers". The model is never told what a cube or the word "red" is; it is only told what your hand did, given what the cameras saw and what the instruction said.

A4.1 Where each recorded field goes

FIELD YOU RECORDED	USED BY ACT AS…	USED BY SMOLVLA AS…	PASSES THROUGH
`observation.images.front`	Input image	Input image → 64 tokens	Frozen vision encoder
`observation.images.wrist`	Input image	Input image → 64 tokens	Frozen vision encoder
`task`	*ignored*	Instruction → word tokens (up to 48)	Frozen language embeddings
`observation.state`	Input (where the arm is)	Input → 1 token	**Trained** state projector
`action` (next 50 frames)	**Target** the model must reproduce	**Target** chunk A for flow matching	Compared with the **trained** action expert's output
`timestamp` , `frame_index` , `episode_index`	Bookkeeping: used to cut each training sample and its 50-step action chunk out of the right episode		Data loader only

A4.2 One training sample, end to end

Follow one frame through one training step. Grey blocks are frozen (pre-trained, never changed by your data); amber blocks are trained by your data. The red dashed line is the learning signal flowing backwards.



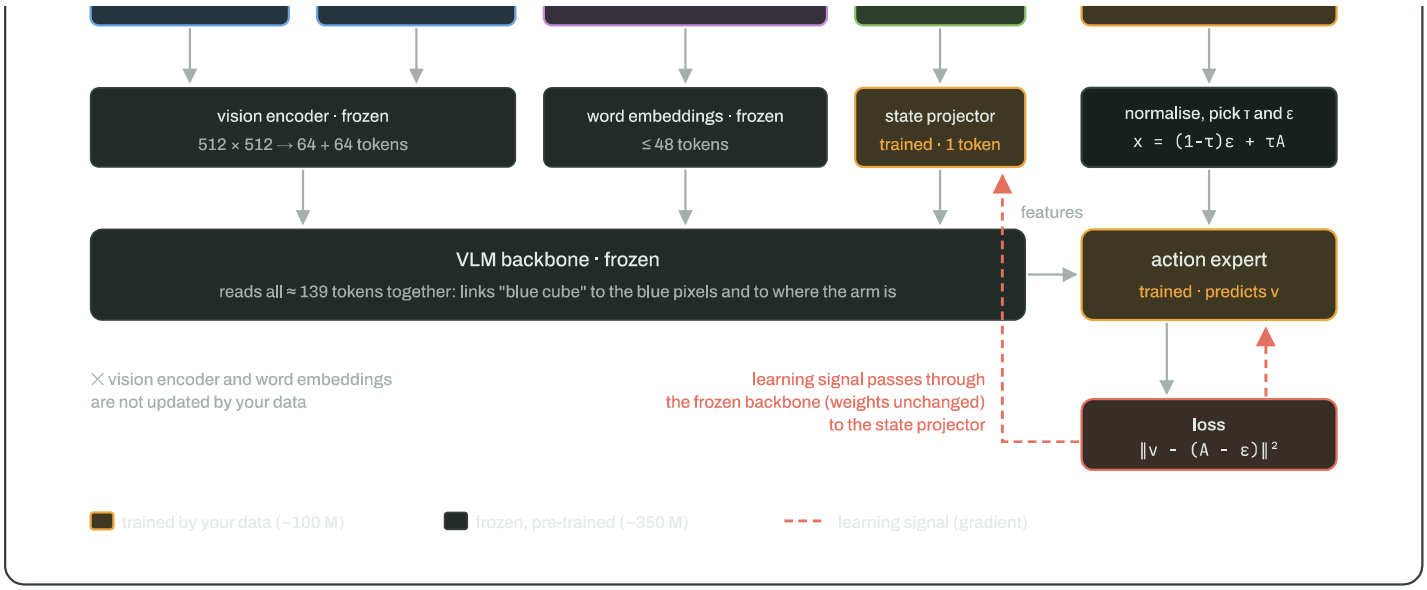


Figure 2e. Where each piece of your data goes in one training step. The frozen parts still run on every step (they turn images and words into features) but their weights never change. Only the amber parts learn.

The same step in words

1. **Cut a sample.** The data loader picks a random episode and frame t. It takes the two camera images and the joint state at frame t, the episode's instruction, and the recorded actions of the next 50 frames as the target chunk A.

2. **Turn inputs into tokens.** Images are resized to 512 × 512 and become 64 tokens each through the frozen vision encoder. The instruction becomes up to 48 word tokens. The state becomes 1 token through the state projector.

3. **Understand.** The frozen backbone reads all the tokens together. Because it was pre-trained on images with text, its features already connect "blue cube" with the blue region of the image. This is where language and vision meet.

4. **Guess the motion.** The target chunk is mixed with random noise (Part A3, Idea 3). The action expert looks at the backbone's features and predicts which direction v would turn that noisy chunk into the real one.

5. **Measure the error and learn.** The loss compares v with the true direction $A - \varepsilon$. The learning signal flows back into the action expert and, through the frozen backbone, into the state projector. Their weights move a little so that next time, for a similar image, sentence and arm pose, the predicted motion is closer to what your hand did.

Repeat 20,000 times with batches of different frames, and the action expert has learned to **read the backbone's understanding of your scene and turn it into motion of your arm.**

A4.3 One episode becomes hundreds of training samples

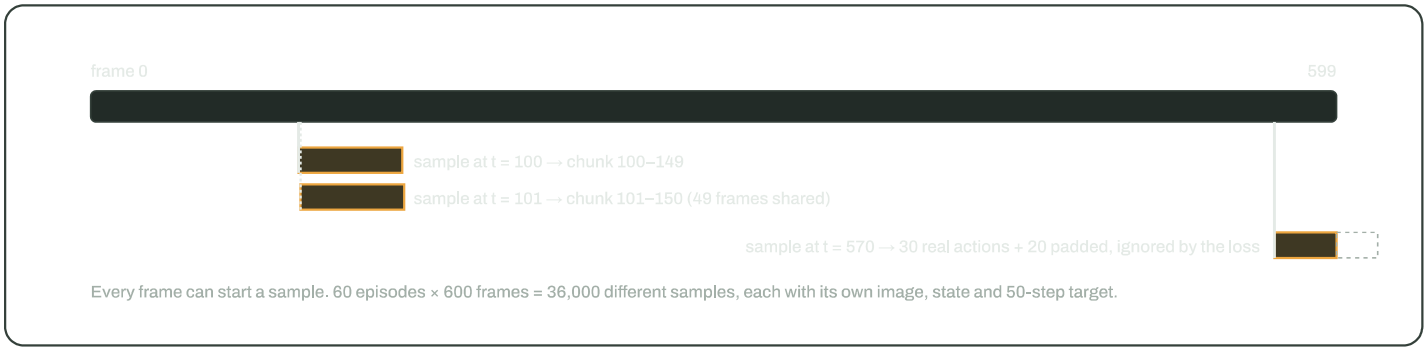


Figure 2f. How 60 episodes become 36,000 samples. Neighbouring samples are nearly identical, which is why consistency between episodes matters more than the raw number of frames.

A4.4 Why the same data is enough

Think of what each part needs to learn, and who teaches it:

WHAT HAS TO BE LEARNED	WHO TEACHES IT	DOES YOUR DATASET NEED TO?
What objects, colours and shapes look like	Web image–caption pairs (stage 1)	No. Only needs images that look like the scene at test time
What the words in the instruction mean	Web text (stage 2)	No. Only needs the instruction text, and needs it to *decide* the task
Which words go with which part of an image	Image + question → answer data (stage 3)	No
Rough reach-grasp-place motion for small arms	481 community robot datasets (stage 4)	No
How *this* arm, calibration, camera view and cube map to joint motion, and which instruction means which	**Your demonstrations** (stage 5)	**Yes. This is all your data teaches**

cube

ACT has to learn the entire bottom-to-top chain from your 50 demos alone, which is why it is limited to one task in one scene. SmolVLA's data only has to fill the last row. The amount of learning your data must do is smaller, even though the model is bigger.

SAME DATASET, TRAINED TWO WAYS	ACT	SMOLVLA
Inputs used	Images, state	Images, state, **instruction**
Vision part	ResNet-18 pre-trained on ImageNet, then fine-tuned on your images	SigLIP encoder, kept frozen
Language part	None	Frozen language model inside the backbone
What is trained	The whole model	Action expert + state projector (≈ 22% of weights)
Target	Chunk of 100 future actions	Chunk of 50 future actions
Loss	Absolute error on actions (plus a small CVAE term)	Flow matching (velocity error)
Can switch task by instruction?	No	Yes, if the data shows both tasks

A4.5 What a VLA needs from the data that ACT did not

The recordings are the same kind, but a VLA puts four extra demands on them:

REQUIREMENT	WHY	SEE
The instruction must decide the action	The language path is only learned if it helps lower the loss; otherwise it is ignored	The robot that does not listen
One exact string per task	Typos create extra "tasks" with fewer examples each	Stage 3.2

Cameras mapped to the model's slots	The pre-trained model expects named views (camera1, camera2…); a missing view is a blind spot	Stage 4, `--rename_map`
Real images like the test scene	The frozen encoder cannot adapt its features to new lighting; the expert only learns to read the features it was shown	Stage 1 lighting, Stage 3.4 variety

KEY IDEA

Your recordings are question-and-answer pairs: "*given these images, this arm pose and this instruction, what should the next 1.7 s of motion be?*", with your demonstration as the answer. A VLA uses the same pairs as ACT. The difference is that the VLA already knows how to see and read, so your answers only have to teach the hands.

STAGE 1 OF 6 SYSTEM PREPARATION

Prepare the rig

Output of this stage: two assembled arms, two rigidly mounted cameras, a marked workspace and a computer that can talk to all four devices.

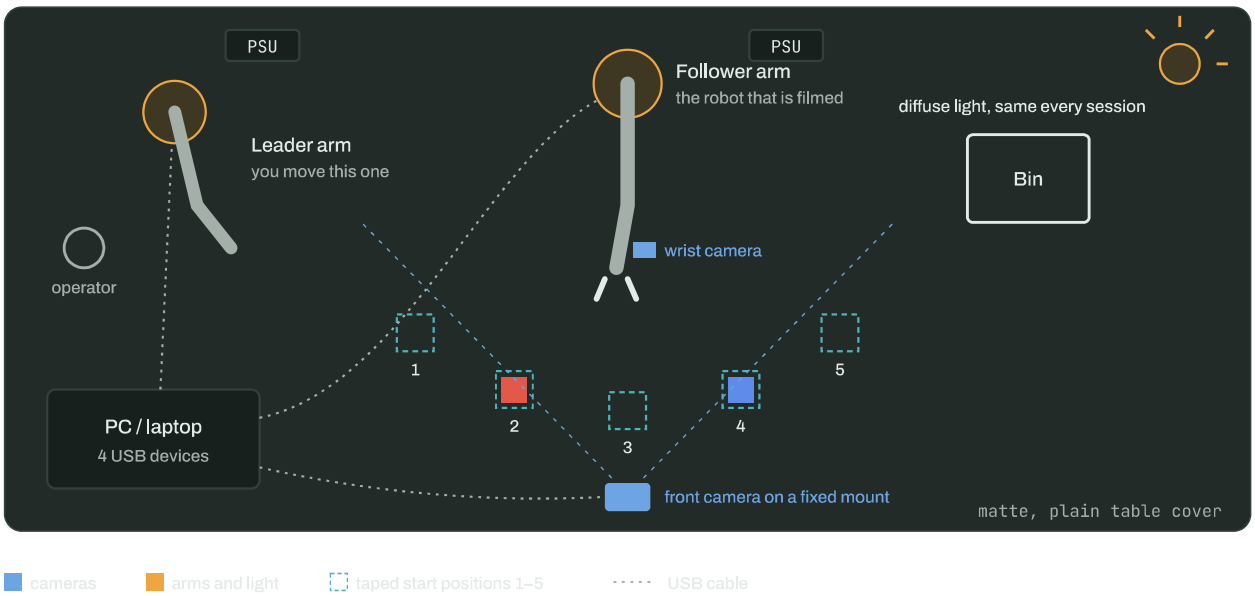


Figure 4. Bench layout, seen from above. Both cubes are on the table in every episode; the instruction decides which one goes in the bin. The front camera must see the gripper, both cubes and the bin at once.

Checklist

ITEM	WHAT TO USE	WHY IT MATTERS
Follower arm	SO-101, 6 × Feetech STS3215 servos, all 1:345 gearing	The robot the model will control.
Leader arm	SO-101 leader, mixed gearing (1:191, 1:345, 1:147) so it is light to move by hand	Your hand motion on it becomes the **action** label.
Power	The supply specified for your servo version: 5 V for the standard 7.4 V servos, 12 V only for the 12 V "Pro" servos	Wrong voltage overheats or destroys servos within minutes.
Front camera	USB webcam, 640 × 480 at 30 fps, on a clamp or 3D-printed mount	Sees the whole scene: where the cubes and the bin are.
Wrist camera	Small USB camera on the gripper, showing both fingertips	Sees the last centimetres of the grasp, where most failures happen.
Workspace	Plain matte cover, 5 taped start positions, a bin, a red and a blue cube	Repeatable positions make the data balanced and the evaluation fair.
Lighting	A fixed lamp; block window light	Sunlight changes through the day; the model sees that as a new scene.
Recording PC	Any laptop with enough USB ports (USB 3 preferred); Ubuntu 22.04/24.04 is the smoothest	Runs teleoperation and saves the video.
Training GPU	16–24 GB VRAM for SmolVLA comfort; a cloud A100 or Colab if your laptop has 8 GB	See the memory maths in Stage 4.

Steps

1. **Assemble and label.** Put a sticker "LEADER" / "FOLLOWER" on each arm and on each USB cable. Half of all beginner errors are the two arms swapped.
2. **Mount the front camera rigidly.** Screw or clamp it; do not rest it on a book. Mark its position on the table with tape so you can tell if it moved.
3. **Mount the wrist camera** so that both fingertips and a cube between them are in view. Route the cable along the arm with slack at every joint, so it never pulls when the wrist rolls.
4. **Tape the five start positions** in an arc the arm can reach comfortably, plus the bin location.
5. **Fix the lighting.** Close blinds, switch on the same lamp each session, and check for strong shadows of the arm across the cubes.
6. **Plug cameras into separate USB ports** (not one hub) so the two video streams do not compete for bandwidth.

What goes wrong here, and how to fix it

CHALLENGE	WHAT YOU SEE	WHY	HOW TO OVERCOME IT
Wrong power supply	Servos hot to touch, smell, then a dead motor	12 V on 7.4 V servos	Check the servo label and kit BOM before plugging in. Unplug power whenever you leave the bench.
Camera moved between sessions	Policy worked yesterday, misses by a few cm today	The model learned pixel positions from the old view	Rigid mount, tape outline, and a reference photo taken from the camera at the start of every session.
Dropped frames	Jerky video, warnings about fps	Two cameras on one USB hub, or high resolution	Separate ports, 640 × 480, 30 fps.
Lighting drift	Success rate falls in the afternoon	Window light changes colours and shadows	Fixed lamp, blinds closed. Later, record some episodes under deliberately varied light.
Wrist cable snags	Camera disconnects mid-episode	Cable too short around the wrist	Leave a service loop; test full wrist roll before recording.

roll



STAGE 2 OF 6 SOFTWARE PREPARATION

Install the software

Output of this stage: a Python environment where `lerobot-teleoperate` moves the follower from the leader while both camera feeds show on screen.

Think of the software as layers. Each layer depends on the one below; most installation problems are a mismatch between two neighbouring layers.

YOUR WORK	Dataset `HF_USER/so101_redblue_bin` and model `HF_USER/smolvla_redblue` on the Hugging Face Hub
LEROBOT CLI	`lerobot-find-port` · `-setup-motors` · `-calibrate` · `-teleoperate` · `-record` · `-train` · `-rollout`
LEROBOT EXTRAS	`feetech` (servo driver) · `smolvla` (model code) · training dependencies
DEEP LEARNING	PyTorch ≥ 2.7 with a CUDA build that matches your GPU (RTX 50-series needs a CUDA 12.8 build)
PYTHON ENV	Python 3.12 (required by LeRobot v0.5 and later), plus `ffmpeg` for video
OPERATING SYSTEM	Ubuntu 22.04 / 24.04 recommended; NVIDIA driver; USB serial access
SEPARATE ENV	Isaac Sim 5.1 / Isaac Lab use Python 3.11. Keep them in their own environment; never install LeRobot into it.

Steps

1. **Create a fresh environment** just for LeRobot.

```
conda create -y -n lerobot python=3.12
conda activate lerobot
conda install -y -c conda-forge ffmpeg
```

2 **Install LeRobot from source** with the servo and SmolVLA extras.

```
git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[feetech,smolvla]"
# v0.6 moved training/dataset dependencies into extras.
# If lerobot-train reports missing packages, add the extra named
# in the installation guide for your version, e.g. ".[feetech,smolvla,training]"
```

3 **Check that PyTorch sees the GPU.**

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

If this prints `False`, reinstall PyTorch using the command for your CUDA version from pytorch.org before going further.

4 **Log in to the Hugging Face Hub** (needs a token with write access).

```
hf auth login
export HF_USER=$(NO_COLOR=1 hf auth whoami | awk -F': *' 'NR==1 {print $2}')
```

5 **Find each arm's port.** Run the tool, unplug the cable when asked, and note the port that disappears. Do it once for each arm.

```
lerobot-find-port
# Linux only, if you get "permission denied":
sudo chmod 666 /dev/ttyACM0 /dev/ttyACM1
```

6 **Save your rig settings in one file** so every later command uses the same values. This single habit prevents most "it worked yesterday" problems.

```
# rig.env   →  load it in every new terminal with:  source rig.env
```

```
export FOLLOWER_PORT=/dev/ttyACM0
export LEADER_PORT=/dev/ttyACM1
export FOLLOWER_ID=so101_follower_A
export LEADER_ID=so101_leader_A
export FRONT_CAM=0
export WRIST_CAM=2
```

7 **Set motor IDs** (one time per arm; follow the prompts and connect one motor at a time).

```
lerobot-setup-motors --robot.type=so101_follower --robot.port=$FOLLOWER_PORT
lerobot-setup-motors --teleop.type=so101_leader  --teleop.port=$LEADER_PORT
```

8 **Calibrate both arms.** Move every joint to the middle of its range, press Enter, then sweep each joint through its full range. The `id` names the calibration file; keep it the same forever.

```
lerobot-calibrate --robot.type=so101_follower --robot.port=$FOLLOWER_PORT --robo
lerobot-calibrate --teleop.type=so101_leader  --teleop.port=$LEADER_PORT  --tele
```

9 **Find the camera indexes** and put them in `rig.env`.

```
lerobot-find-cameras opencv
```

10 **Teleoperate with both cameras on screen.** If this works smoothly, Stage 2 is done.

```
lerobot-teleoperate \
  --robot.type=so101_follower --robot.port=$FOLLOWER_PORT --robot.id=$FOLLOWER_I
  --robot.cameras="{ front: {type: opencv, index_or_path: $FRONT_CAM, width: 640
  --teleop.type=so101_leader --teleop.port=$LEADER_PORT --teleop.id=$LEADER_ID \
  --display_data=true
```

ON WINDOWS LAPTOPS

Ports appear as `COM3`, `COM4` instead of `/dev/ttyACM*`, and you set variables with `set` (cmd) or `$env:` (PowerShell) instead of `export`. Teleoperation and recording can work on Windows, but video encoding and keyboard shortcuts give more trouble; keep one Ubuntu machine in the lab as the reference setup.

What goes wrong here, and how to fix it

CHALLENGE	WHAT YOU SEE	WHY	HOW TO OVERCOME IT
Python version clash	Install errors mentioning `requires-python`	LeRobot ≥ v0.5 needs Python 3.12; Isaac Sim 5.1 needs 3.11	Two separate environments. Never mix them.
GPU not used	`cuda.is_available()` is False; training is very slow	CPU-only PyTorch, or a CUDA build too old for the GPU	Install the PyTorch wheel matching your driver (cu128 for RTX 50-series).
Ports swap after reboot	Follower stays still, leader twitches	The OS numbers USB devices in plug-in order	Re-run `lerobot-find-port` at the start of each session, or add a Linux udev rule that gives each arm a fixed name.
Calibration mismatch	Follower jumps to a strange pose when started	A different `--robot.id` loaded a different (or no) calibration	Use the IDs from `rig.env` in every command.
Camera index changes	Front and wrist images swapped	Indexes depend on plug-in order	Check with `lerobot-find-cameras` each session; look at the preview before recording.
Version drift	A tutorial's flag is "unrecognised"	LeRobot renames options between releases (v0.6 renamed several)	Run `lerobot-record --help` and trust it over any web page, including this one.

Collect the data

Output of this stage: a LeRobot dataset of about 60 clean episodes, balanced across two instructions and five start positions, checked by eye.

3.1 What one recorded frame contains

LeRobot records 30 frames per second. Every frame stores the same fields:

`observation.images.front`
480 × 640 colour image (stored inside an MP4)

`observation.images.wrist`
480 × 640 colour image

`observation.state`
6 numbers: where the follower's joints *are*

`action`
6 numbers: where the leader told the follower to *go*

`task`
the instruction text for this episode

`timestamp` · `frame_index` · `episode_index`
bookkeeping

State versus action is the most important pair. The state is what the robot measured; the action is what your hand on the leader commanded at that instant. The model learns "given what I see and where I am, where should I go next".

On disk the dataset uses the LeRobotDataset format: tables of numbers in Parquet files, video per camera in MP4, and a `meta/` folder.

```
~/.cache/huggingface/lerobot/HF_USER/so101_redblue_bin/
├── meta/
│   ├── info.json        # fps, features, camera names, robot type
│   ├── stats.json       # μ and σ for every feature (Part A3, Idea 2)
│   ├── tasks.parquet    # the instruction strings
│   └── episodes/        # where each episode starts and ends
├── data/                # state, action, indexes (Parquet)
└── videos/
    ├── observation.images.front/
    └── observation.images.wrist/
```

3.2 Design the dataset before you record

Data design is where VLA projects succeed or fail. Three rules:

1. **Cover the space evenly.** The SmolVLA authors recorded 5 cube positions × 10 episodes. A 25-episode version of the same task performed poorly. Start at 50 or more.

2. **One strategy per situation.** Always approach from the same side, always grasp the same way. Two styles for the same situation is Figure 3's averaging trap in your own data.

3. **Make the instruction necessary.** Put both cubes on the table in every episode and let the sentence decide which one moves. This is the central failure mode of VLA training, explained next.

CENTRAL FAILURE MODE

The robot that does not listen

A network only learns to use an input if that input helps it reduce the loss. If every episode in your dataset says "Put the red cube in the bin", the sentence is identical in all 36,000 frames. It carries no information about what the arm should do, so training gives the model no reason to look at it. The result is a VLA that **ignores language**: change the instruction to "blue" at test time and it still reaches for red, or for whichever cube is nearest.

A simple test, before you record anything: **cover up the instruction. Could a human still do every episode perfectly from the images alone?** If yes, the model will learn to ignore the instruction.

DATASET DESIGN	IMAGES ALONE TELL YOU THE TARGET?	DOES THE MODEL NEED THE WORDS?
One cube, one instruction	Yes	No — it will ignore them
Red and blue cubes, instruction always "red"	Yes (it is always red)	No — it will ignore them
Red and blue cubes, "red" in half the episodes and "blue" in the	No (a 50/50 guess)	Yes — it must read them

and blue in the other half

Fix: paired scenes (both cubes, every episode), equal numbers of each instruction, and the cubes' left/right placement swapped across episodes so that "red" cannot be learned as "the left one". You will test for this failure with the instruction-swap test in Stage 6.

A recording plan that satisfies all three rules

BLOCK	INSTRUCTION (EXACT STRING)	TARGET CUBE AT	OTHER CUBE AT	EPISODES
R1–R5	`Put the red cube in the bin`	each of positions 1–5	a different position, varied	5 × 6 = 30
B1–B5	`Put the blue cube in the bin`	each of positions 1–5	a different position, varied	5 × 6 = 30
Total				60

Use one exact instruction string per task and reuse it at deployment. If you want the robot to understand other phrasings ("drop the red block into the container"), you must also record with those phrasings.

CALCULATOR **Data budget**

How much data you will collect, how long recording takes, and how many times training will see each frame.

EPISODES	AVG. SECONDS PER EPISODE	RESET SECONDS	FPS
60	20	15	30

CAMERAS	TRAINING STEPS	BATCH SIZE
2	20000	64

36,000	72,000	20 min	35 min
frames = episodes × seconds × fps	camera images = frames × cameras	of demonstration	recording time incl. resets (plus breaks)

35.6×

times each frame is seen =
steps × batch ÷ frames

Each frame is seen about 36 times. That is normal for fine-tuning on a small dataset; watch for memorising (Stage 4).

3.3 Record

1. **Record the first block.** Episode length 25 s gives you room to finish; reset time 15 s lets you place the cubes for the next episode.

```
source rig.env
lerobot-record \
  --robot.type=so101_follower --robot.port=$FOLLOWER_PORT --robot.id=$FOLLOWER_I
  --robot.cameras="{ front: {type: opencv, index_or_path: $FRONT_CAM, width: 640
  --teleop.type=so101_leader --teleop.port=$LEADER_PORT --teleop.id=$LEADER_ID \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/so101_redblue_bin \
  --dataset.single_task="Put the red cube in the bin" \
  --dataset.num_episodes=6 \
  --dataset.episode_time_s=25 \
  --dataset.reset_time_s=15
```

2. **Use the keyboard while recording.** → (or `n`) ends the episode early when the cube is in the bin; ← (or `r`) throws away the episode and records it again; `Esc` (or `q`) stops, encodes the videos and uploads.

3. **Add the next blocks to the same dataset** with `--resume=true`. `num_episodes` now means "how many more", and you change the instruction for the blue blocks.

```
lerobot-record ...same robot/teleop/camera flags... \
  --dataset.repo_id=${HF_USER}/so101_redblue_bin \
  --dataset.root=~/.cache/huggingface/lerobot/${HF_USER}/so101_redblue_bin \
  --resume=true \
  --dataset.single_task="Put the blue cube in the bin" \
  --dataset.num_episodes=6
```

4. **Alternate red and blue blocks** through the session rather than all red first. If the light or your skill changes over two hours, both tasks are affected equally.

5. **Look at your data.** Replay a few episodes per block and check that the action traces are smooth and the images are sharp.

```
lerobot-dataset-viz --repo-id ${HF_USER}/so101_redblue_bin --episode-index 0
```

A GOOD DEMONSTRATION	A BAD DEMONSTRATION
Smooth, steady speed from start to finish	Hesitates, stops halfway, starts again
One clean grasp	Misses, re-grasps, nudges the cube
Same approach direction as all other episodes	Sometimes from the left, sometimes from above
Ends with the cube in the bin and the arm back at a rest pose	Ends with the arm wherever it stopped
The cube is visible in the front camera the whole time	Your hand or the arm hides the cube

You can only teach what is in the camera images. A good check: could you do the task yourself while looking only at the two camera feeds? Section 3.4 goes through the grasp and lift in detail.

OPTIONAL · SIMULATED DATA

The imitation-learning lab used a leader arm to drive the simulated SO-101 in Isaac Lab. Those episodes can be exported in LeRobot format and added to the real ones. Rendered images never look exactly like your webcam, so keep real episodes as the core of the dataset and treat simulation as extra variety, then check on the real arm whether it helped.

3.4 Demonstrating a good grasp and lift

Most failed SO-101 policies fail in the same two seconds: closing the gripper and lifting the cube. That is also where demonstrations differ most from one operator to the

next. The model can only learn a clean grasp if every episode shows the same clean grasp, so this section breaks the pick into phases, shows good and bad examples of each, and gives you checks you can run on your data.

The eight phases of a good pick

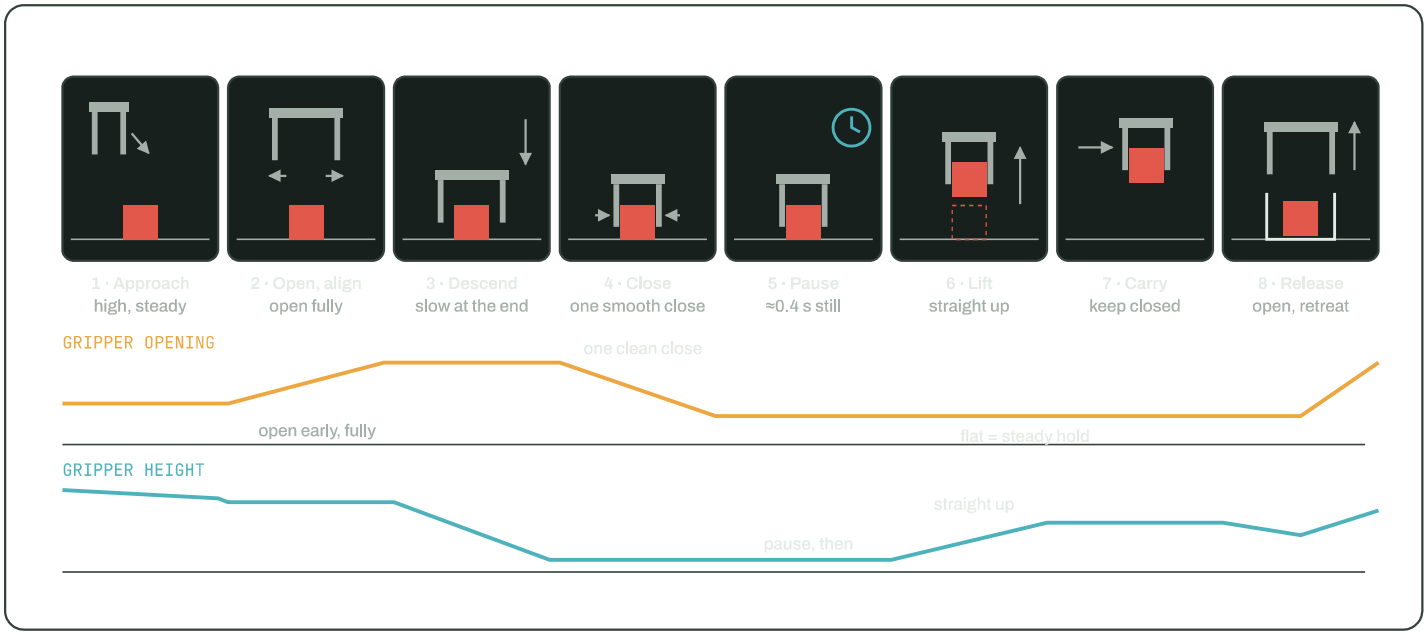


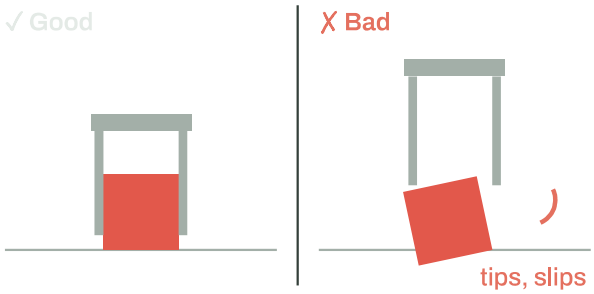
Figure 4b. A good pick, phase by phase, with the gripper and height signals you will see when you replay the episode. Each phase panel sits above its part of the timeline.

PHASE	DO THIS	WHY THE MODEL NEEDS IT
1 · Approach	Move above the cube at a steady speed, staying high enough to clear it	A repeatable path the model can copy from any start position
2 · Open & align	Open the gripper fully *before* descending; turn the wrist so the fingers are parallel to the cube's faces	Separates "get ready" from "go down", so each phase has one clear action
3 · Descend	Go straight down, slowing for the last 2–3 cm, until the fingertips are level with the lower half of the cube	Slow motion near contact gives more frames of the critical moment and less follower lag
4 · Close	Close in one smooth movement until the fingers stop on the cube	One close per episode; no hesitation or re-try in the labels
5 · Pause	Hold still for about 0.4 s	Lets the grip settle, and shows the model clearly that lifting comes *after* closing
6 · Lift	Lift straight up 3–5 cm before moving	Clears the table without dragging; easy

	sideways	for the model to time
7 · Carry	Move to above the bin at steady speed, gripper held closed	A flat gripper signal teaches "keep holding while moving"
8 · Release & retreat	Lower until the cube is just above the bin, open, then lift away before going to rest	A gentle drop, and a clear end to the episode

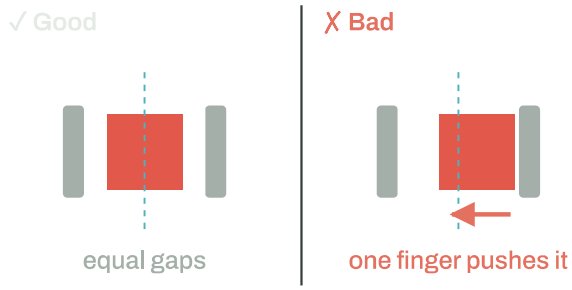
Good and bad, side by side

Each card shows a good demonstration on the left and a common bad one on the right, with the physical reason and what the model would learn from it.



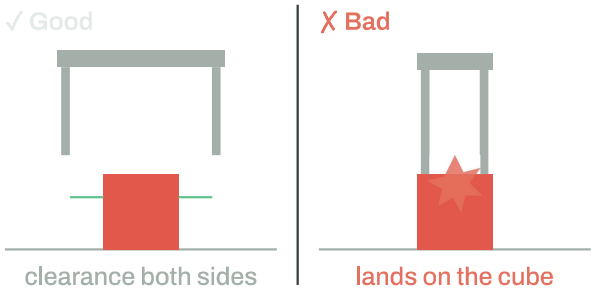
Grip low, not on the top edge

A grip near the middle or lower half holds the cube on two full faces. A grip on the top edge touches a thin strip, so the cube can pivot out when it accelerates upward. If some demos grip high and some low, the model learns a height in between.



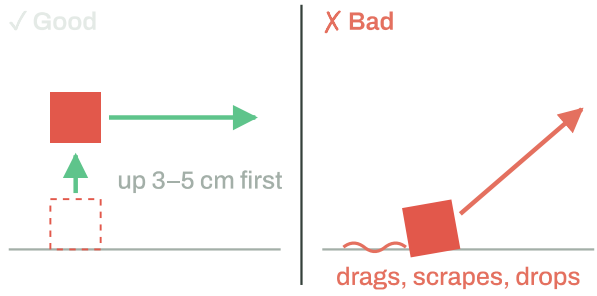
Centre the cube between the fingers

Off-centre, one finger touches first and slides the cube across the table before the other arrives. Check the wrist camera: the cube should sit in the middle of the gap before you close.



Open fully before going down

Open width should be the cube width plus a margin on each side (e.g. a 25 mm cube with

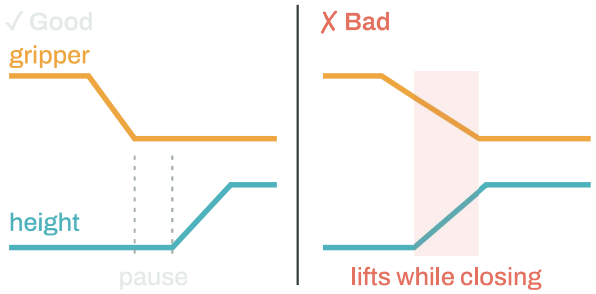


Lift straight up, then move

A diagonal pull drags the bottom of the cube on the table. Friction pulls it out of the grip,

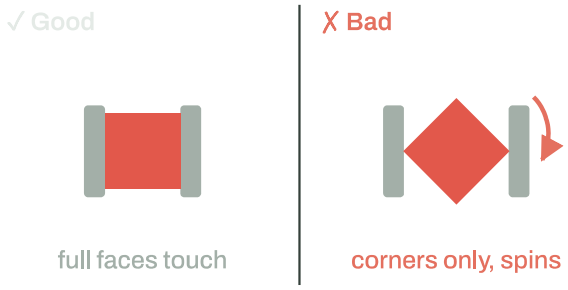
10 mm margins needs 45 mm). Opening *while* descending mixes two actions and sometimes hits the cube; the model then learns a risky timing.

and the follower lags behind the leader most during fast combined moves.



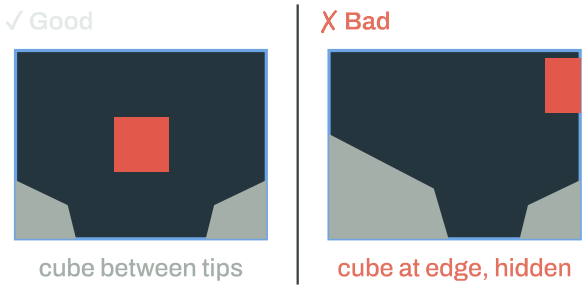
Close, pause, then lift

About 0.4 s is 12 frames at 30 fps. Those frames show the model a clear rule: lift only once the fingers have stopped. When closing and lifting overlap, some demos lift early and some late, and the policy learns to lift before the grip is secure.



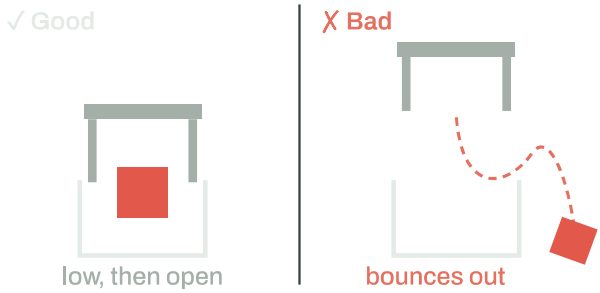
Turn the wrist to match the cube

Fingers on two flat faces resist twisting; fingers on two corners let the cube rotate out. When you later add rotated cubes, always roll the wrist to match, so wrist roll in your data follows the cube's angle.



Keep the cube in the wrist camera

The wrist camera is the model's main view of the last few centimetres. If the cube is at the edge or behind a finger, the model has to guess. Watch the wrist feed on screen while you record, not only the arm.



Lower before releasing

Dropping from high up lets the cube bounce off the rim and out of the bin, and the episode ends with a different result each time. Lower until the cube is just inside the bin, open, then lift away. A consistent release also gives the model a clear signal for when the task is finished.

The physics behind a secure grip

Two fingers hold the cube by friction. Each finger presses with force F, and friction on

each face can be at most μF. To lift, friction must beat the cube's weight plus the extra force needed to accelerate it upward:

MINIMUM GRIP FORCE PER FINGER

$$2\,\mu\,F \geq S \cdot m\,(g + a) \;\Rightarrow\; F \geq \frac{S \cdot m\,(g + a)}{2\,\mu}$$

m = cube mass, g = 9.81 m/s², a = upward acceleration while lifting, μ = friction coefficient between finger and cube, S = safety factor.

CALCULATOR How hard must the fingers squeeze?

CUBE MASS (G)	FRICTION M	LIFT ACCELERATION (M/S²)	SAFETY FACTOR
30	0.5	2	2

0.29 N	0.35 N	0.71 N
weight m·g	force to lift, m(g + a)	minimum squeeze per finger

About 0.71 N per finger, roughly the weight of a 72 g object. Halve μ (a smooth, shiny cube) and you need twice the squeeze.

What this means for demonstrations:

- **Lift gently.** A jerky lift raises a, so the same grip that held during a slow lift can fail during a fast one.
- **Grip faces, not corners or edges**, which keeps contact area and μ high (the first and sixth cards above).
- **Choose easy objects first.** Light, matte, rubbery or 3D-printed cubes have high μ; smooth glossy plastic does not.
- **Close firmly, but do not keep squeezing.** The follower's gripper servo stops on the cube while the leader keeps commanding "more closed". A small, consistent squeeze is good; a hard squeeze heats the servo, and an inconsistent squeeze gives the model inconsistent gripper labels.

Speed: why slow near contact gives better data

Speed: why slow near contact gives better data

Two things get worse as you move faster: the image smears during each camera exposure (motion blur), and the follower falls further behind the leader (lag), so the action label and the actual pose disagree.

BLUR AND LAG

$$\text{blur (px)} = \text{speed} \times \text{exposure time} \times \text{pixels per mm} \qquad \text{lag distance} = \text{speed} \times \text{follower lag}$$

CALCULATOR **Demo speed check**

GRIPPER SPEED (MM/S)	EXPOSURE (MS)	IMAGE SCALE (PX/MM)	FOLLOWER LAG (MS)
300	16.7	2	100

10.0 px
motion blur in the image

30 mm
follower behind the leader

Fast: 10.0 px of blur and 30 mm of lag. Fine for the open-air approach, too much near the cube. Try 50 mm/s for the last few centimetres.

The exposure, image scale and 100 ms lag are illustrative; the wrist camera has a larger px/mm than the front camera because it is closer. You can estimate your own lag by plotting `action` against `observation.state` for one joint and measuring the time shift between them.

Check your grasps after recording

The gripper column of the action is the quickest quality check. In a good episode it changes state exactly twice: open → closed at the grasp, closed → open at the bin. A failed grasp and re-try shows up as extra changes.

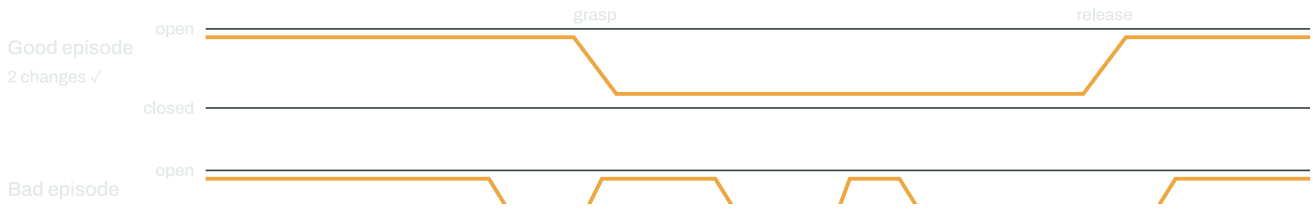




Figure 4c. The gripper signal tells you which episodes to delete. The bad episode would teach the policy that missing and re-trying is normal.

This short script lists suspicious episodes. It reads the dataset's Parquet files directly; check the folder layout and column names against your LeRobot version.

```
# check_grasps.py — flag episodes whose gripper does not open/close exactly once each
import glob, os
import numpy as np, pandas as pd

root = os.path.expanduser("~/.cache/huggingface/lerobot/HF_USER/so101_redblue_bin")
files = sorted(glob.glob(f"{root}/data/**/*.parquet", recursive=True))
df = pd.concat(pd.read_parquet(f, columns=["episode_index", "frame_index", "action"])

for ep, g in df.sort_values(["episode_index", "frame_index"]).groupby("episode_index")
    grip = np.stack(g["action"].to_numpy())[:, 5]           # column 5 = gripper
    if grip.max() - grip.min() < 5:                         # gripper hardly moved
        print(f"episode {ep}: gripper never closed"); continue
    closed = (grip < (grip.max() + grip.min()) / 2).astype(int)
    changes = int(np.abs(np.diff(closed)).sum())
    if changes ≠ 2:
        print(f"episode {ep}: {changes} gripper changes, replay and check")
```

The script does not care whether a larger number means open or closed; it only counts crossings of the midpoint. Replay every flagged episode with `lerobot-dataset-viz` before deleting it.

Operator habits that make the biggest difference

HABIT	HOW
Rehearse first	Do 5 practice picks without recording, until every one looks the same. Throw away the first few recorded episodes of each session if they feel clumsy.
Watch the follower and the wrist feed	Not the leader in your hand. The camera images are what the model sees.
Hold the leader near the	Two hands: one guiding the wrist, one on the gripper handle. Gives fine

wrist	control near the cube.
Use a rhythm	Count in your head: "approach, open, down, close, one, lift". Similar timing across episodes is part of consistency.
Redo, don't rescue	If the grasp misses, press ← and record the episode again. Do not nudge the cube or re-grasp on camera.
Keep your hands out of view	Reset the cube during the reset period only. A hand in the frame is a feature the robot will never see at test time.
Swap operators between blocks, not within	If several learners record, agree on the technique together and check each other's first episodes.

Adding variety without breaking consistency

Variety helps the policy generalise, but only after the basic grasp is reliable. Add one kind of variation at a time and keep the grasp technique the same throughout.

ROUND	WHAT VARIES	WHAT STAYS THE SAME	MOVE ON WHEN
1 · Basics	The 5 taped positions	Cube square to the arm, same cube, same lighting	Policy succeeds at taught positions
2 · Placement	Positions between the tapes; small offsets	Orientation, cube, lighting	Succeeds between taught positions
3 · Orientation	Cube rotated 0°, 20°, 45°; wrist roll follows	Cube, lighting	Grips rotated cubes on faces, not corners
4 · Appearance	Another cube size or material; lamp position	Grasp technique	Robust under the Stage 6 lighting and distractor tests

WHAT ABOUT RECOVERY DEMOS?

Robots that can recover from a slip are more robust, and some teams deliberately record recoveries. Do this only after the basic dataset works, as a separate, consistent block (always the same recovery motion). Accidental re-grasps mixed into normal episodes do more harm than good.

into normal episodes do more harm than good.

What goes wrong here, and how to fix it

CHALLENGE	WHAT YOU SEE LATER	WHY	HOW TO OVERCOME IT
Language ignored	Picks the same cube whatever you say	The instruction never decided the action	Paired-scene design, balanced instructions (box above).
Too few episodes	Works only at the exact taught spots	The model has not seen enough variety	≥ 50 episodes, ≥ 10 per position (or 6 per position per task).
Inconsistent strategy	Arm hesitates midway, or aims between two paths	Averaging between different styles	Agree on one approach before recording; one operator per dataset if possible.
Sloppy grasps	Grips the top edge, lifts too early, drops the cube	Grasp height and timing varied across demos	Follow the eight phases and the good/bad cards in 3.4; run the gripper check.
Failed demos kept	Robot copies your mistakes (re-grasps, nudging)	Bad episodes are training data too	Press ← to redo during recording; delete bad episodes afterwards with LeRobot's dataset editing tool.
Recalibrated mid-dataset	A constant offset at contact	Same pose, different numbers (Idea 2)	Calibrate once per dataset. If you must recalibrate, start a new dataset.
Too much variety too soon	Nothing works well	The task became harder than the data can teach	Get reliable grasping first, then add new positions, lighting or distractors step by step.

STAGE 4 OF 6 TRAINING

Train (fine-tune SmolVLA)

Output of this stage: two or three saved checkpoints of a SmolVLA model fine-tuned on

your dataset, and a training log.

You do not train a VLA from zero. You start from `lerobot/smolvla_base`, which has already been pre-trained, and **fine-tune** it: a short extra training run that adapts it to your arm, your cameras and your two tasks.

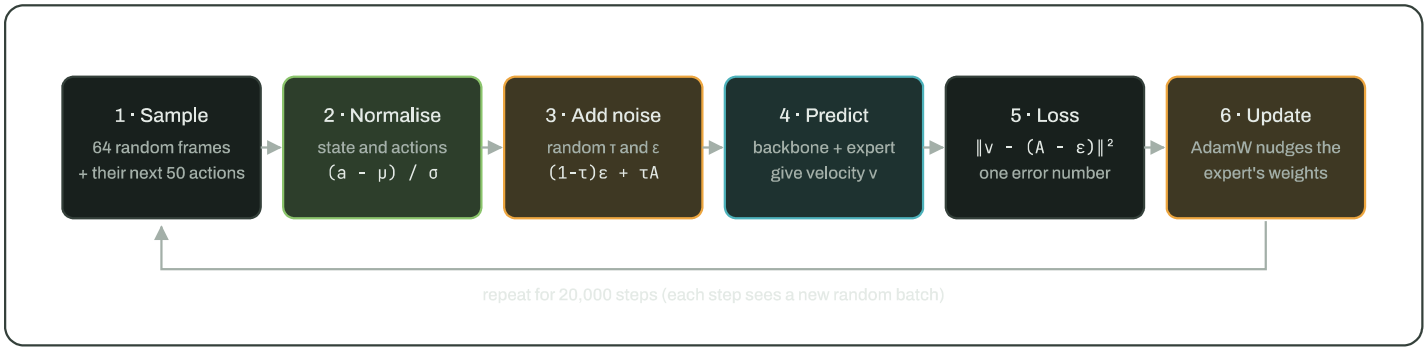


Figure 5. One training step. Steps 3 to 5 are exactly the flow-matching recipe from Part A3. The frozen backbone still runs in step 4; it just does not get updated in step 6.

Choose where to train: the memory maths

Training needs GPU memory for the weights, and, for every weight that is being trained, its gradient and two optimiser values. A rule of thumb for the AdamW optimiser:

RULE OF THUMB, BEFORE ACTIVATIONS

memory ≈ trainable × 16 bytes + frozen × (2 to 4) bytes

SET-UP	TRAINABLE	FROZEN	ESTIMATE	VERDICT
SmolVLA, default (expert only)	~100 M → 1.6 GB	~350 M → 0.7–1.4 GB	≈ 2–3 GB	Fits a lab GPU; batch size decides the rest
π0, full fine-tune	3.3 B → 53 GB	—	≈ 53 GB	Needs an 80 GB-class GPU
π0 with LoRA (~1% trained)	~33 M → 0.5 GB	3.3 B → 6.6 GB	≈ 7 GB	Possible on 24 GB once activations are added

Activations (the intermediate results kept for back-propagation) come on top and grow with batch size and image count. That is why `--batch_size` is your main dial

when you run out of memory. These are estimates for planning, not measurements; watch `nvidia-smi` on your own machine.

Steps

1. **Start the fine-tune.** These are the settings recommended in the SmolVLA documentation.

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/so101_redblue_bin \
  --batch_size=64 \
  --steps=20000 \
  --save_freq=5000 \
  --output_dir=outputs/train/smolvla_redblue \
  --job_name=smolvla_redblue \
  --policy.device=cuda \
  --policy.repo_id=${HF_USER}/smolvla_redblue \
  --wandb.enable=true
```

2. **If training stops with a camera or feature-name mismatch**, map your camera names onto the names the base model expects, and mark unused camera slots as empty:

```
  --rename_map='{"observation.images.front":"observation.images.camera1","observ
  --policy.empty_cameras=1 \
```

3. **If you run out of GPU memory**, lower the batch to 16 or 8. Smaller batches give noisier updates, so increase `--steps` in proportion if you can afford the time.

4. **Watch the loss curve** in Weights & Biases (or the terminal log every few hundred steps). Read Figure 6 before you judge it.

5. **Check the checkpoint folder** is complete before deploying: `outputs/train/smolvla_redblue/checkpoints/last/pretrained_model/` should contain `config.json`, `model.safetensors`, `train_config.json` and the pre- and post-processor files (these carry μ and σ).

FLAG	MEANING	BEGINNER ADVICE

FLAG	MEANING	BEGINNER ADVICE
`--policy.path`	Which pre-trained model to start from	Always `lerobot/smolvla_base` for this lab
`--dataset.repo_id`	Your dataset on the Hub (or local cache)	Same name you recorded with
`--batch_size`	Frames per training step	64 on a large GPU; 8–16 on a laptop
`--steps`	Number of updates	20,000 is the documented baseline
`--save_freq`	Save a checkpoint every N steps	5,000 gives you four to compare
`--policy.repo_id`	Where to upload the trained model	Lets the deployment PC download it
`--wandb.enable`	Online loss plots	Very helpful; needs a free W&B account

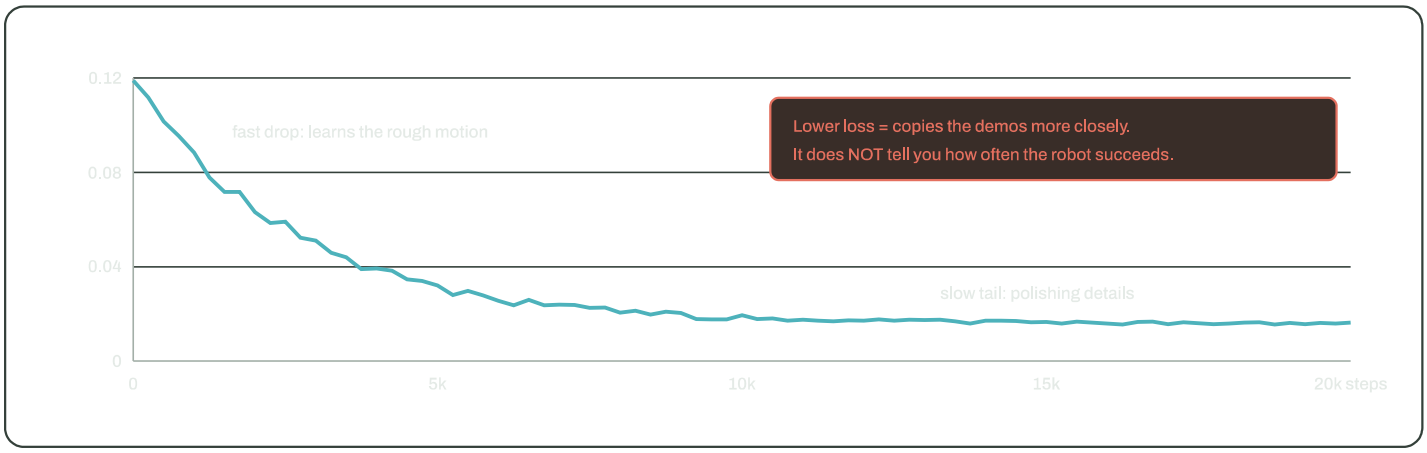


Figure 6. What a healthy loss curve looks like (illustrative shape). One published SmolVLA run on an SO-101 pick-and-place dataset went from a mean loss of about 0.10 in the first 4,000 steps to about 0.018 in the last 4,000. Only the real arm can tell you whether the task works; that is Stage 6.

4.1 The knobs you can turn

A training run is controlled by a handful of **hyperparameters**: settings you choose before training, as opposed to the *parameters* (weights) that training learns. SmolVLA ships with a tested set of defaults, stored in the policy's `config.json`. Start with them, and change one at a time for a reason.

KNOB (CLI FLAG)	SMOLVLA DEFAULT	WHAT IT DOES	WHEN TO CHANGE IT
`--policy.optimizer_lr`	`1e-4`	Peak learning rate: how big each weight update is	Lower to 5e-5 or 3e-5 if the loss is spiky or the batch is small
`--policy.scheduler_warmup_steps`	`1,000`	Steps to ramp the learning rate up from almost 0	Keep about 3–5% of total steps
`--policy.scheduler_decay_steps`	`30,000`	Step at which the learning rate reaches its floor	Set it close to your total `--steps`
`--policy.scheduler_decay_lr`	`2.5e-6`	The floor the learning rate decays to	Rarely
`--batch_size`	`64` (docs)	Frames averaged per update; bigger = smoother	As large as GPU memory allows
`--steps`	`20,000` (docs)	Number of updates	More if the learning rate had not finished decaying; compare earlier checkpoints if memorising
`--policy.optimizer_grad_clip_norm`	`10`	Caps the size of one update if a batch produces a huge gradient	Lower (e.g. 1) if you see loss spikes or NaN
`--policy.optimizer_weight_decay`	`1e-10`	Pulls weights slightly towards zero; effectively off here	Advanced overfitting experiments only
`--policy.optimizer_betas`	`(0.9, 0.95)`	How much AdamW averages past gradients	Leave alone
`--policy.freeze_vision_encoder` / `train_expert_only`	`true` / `true`	Which parts are trained (Part A2)	Only after data and schedule are ruled out; needs more memory and data
`--policy.use_peft`	`false`	LoRA: train small	Alternative to unfreezing

		add-on matrices instead	when memory is tight
`--dataset.image_transforms.enable`	`false`	Random brightness, contrast, colour and sharpness changes to training images	To make the policy tolerate lighting changes
`--seed`	`1000`	Fixes the random choices so a run can be repeated	Keep fixed while comparing settings

Defaults read from SmolVLA's published `config.json` and `train_config.json`. With `--policy.path`, any field can be overridden as `--policy.<field>=value`; the optimiser and scheduler come from the policy's own preset unless you turn that off.

4.2 Learning rate and the AdamW optimiser

Each training step computes the **gradient** g: for every trainable weight, which direction would lower the loss and how strongly. The **optimiser** decides how far to actually move. SmolVLA uses **AdamW**, which keeps two running averages per weight:

ADAMW, SIMPLIFIED (ONE WEIGHT)

$$m \leftarrow 0.9\,m + 0.1\,g \quad \text{(average direction, "momentum")}$$

$$v \leftarrow 0.95\,v + 0.05\,g^2 \quad \text{(average size)}$$

$$w \leftarrow w - \mathrm{lr} \cdot \frac{m}{\sqrt{v} + \varepsilon} - \mathrm{lr} \cdot \lambda \cdot w \quad (\lambda = \text{weight decay})$$

The ratio $m/\sqrt{v}$ is close to ±1 when the gradient points the same way step after step, and close to 0 when it keeps flipping. That gives AdamW two useful behaviours:

GRADIENT PATTERN FOR ONE WEIGHT	M	√V	M / √V	STEP (LR = 1E-4)	MEANING
Steady, small: +0.02 every step	0.02	0.02	1.0	1e-4	Moves at full speed
Steady, large: +2.0 every step	2.0	2.0	1.0	1e-4	Same speed: the size of the gradient does

				gradient does not matter
Noisy: +0.02, −0.02, +0.02, …	≈ 0.001	0.02	≈ 0.05	≈ 5e-6	Barely moves: disagreement cancels out

Values are the long-run averages; the real optimiser also corrects the averages at the start of training (bias correction).

So the **learning rate is roughly the distance each weight can move per step**. Too high and the weights jump past the bottom of the valley and bounce; too low and they crawl. And when your demonstrations disagree with each other, the gradients disagree too, and the model stops moving. Keep that last point in mind for the case study.

ANALOGY

Walking down a foggy hill to the lowest point. The gradient is the slope under your feet, the learning rate is your stride length, and momentum is trusting the direction you have been walking in for the last few steps. Long strides get you down fast but make you overshoot the bottom; short strides are precise but slow. That is why the stride is shortened as training goes on.

4.3 The learning-rate schedule

SmolVLA does not keep the learning rate fixed. It **warms up** linearly over the first 1,000 steps (so the pre-trained weights are not hit with full-size updates while AdamW's averages are still empty), then follows a **cosine decay** down to a small floor:

AFTER WARM-UP (T = STEP, T = DECAY STEPS)

$$\mathrm{lr}(t) = \mathrm{lr}_{\min} + (\mathrm{lr}_{\max} - \mathrm{lr}_{\min}) \cdot \tfrac{1}{2}\left(1 + \cos(\pi \cdot \min(t, T) / T)\right)$$

STEP (T = 30,000)	LEARNING RATE	% OF PEAK
500 (warming up)	5.0e-5	50%
1,000	1.0e-4	100%

10,000	7.6e-5	76%
15,000	5.1e-5	51%
20,000 (end of a 20k run)	2.7e-5	27%
30,000	2.5e-6	2.5%

Notice the highlighted row. With the default `scheduler_decay_steps=30000` and `--steps=20000`, the run stops while the learning rate is still at 27% of its peak: training ends before the slow, precise "polishing" phase. Newer LeRobot versions detect this and shrink the schedule to fit your step count; older ones do not. Check the `lr` value printed in your training log at the last step.

CALCULATOR **Learning-rate schedule**

PEAK LR: 0.0001

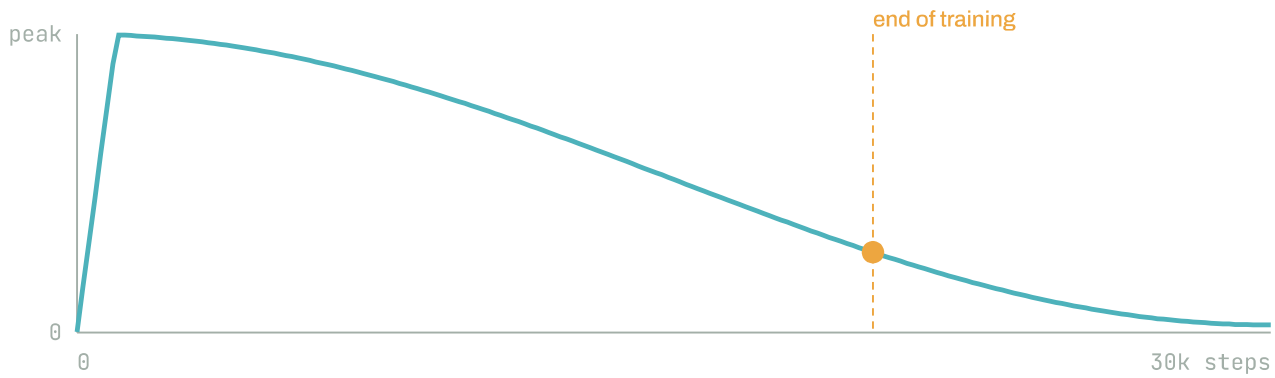
FINAL LR: 0.0000025

WARM-UP STEPS: 1000

DECAY STEPS: 30000

TRAINING STEPS: 20000

SHRINK SCHEDULE TO FIT: ☐ newer LeRobot



2.7e-5 learning rate at the last step

27% of the peak

15,250 step where lr drops to half

Training stops while the learning rate is still 27% of its peak, before the low-LR polishing phase. Set decay steps close to 20,000, or train to 30,000 steps.

4.4 Reading the loss curve

4.4 Reading the loss curve

The training loss jumps around from step to step, because every batch holds different frames and flow matching draws new random noise and τ for each one (Part A3). Read the *smoothed trend* over a few thousand steps, not single values. Four shapes cover most runs:

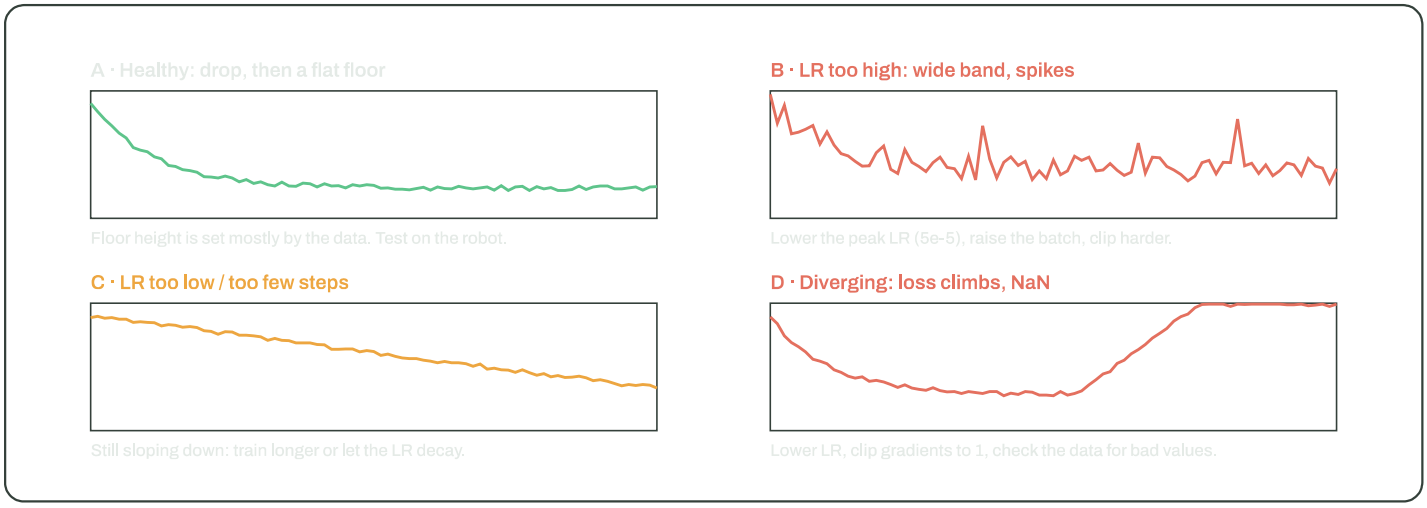


Figure 6b. Four loss shapes and what to do about each (illustrative curves). A plateau like 0.08 after 20,000 steps looks like shape A; the question is whether its floor is as low as your data allows.

Why the loss stops at a floor: the data decides

Suppose that in five demonstrations the robot sees almost the same image, but you closed the gripper at slightly different moments. The model sees one situation and five different "right answers". The best it can do is predict their average, and the spread that is left over is a **loss floor** no amount of training can remove.

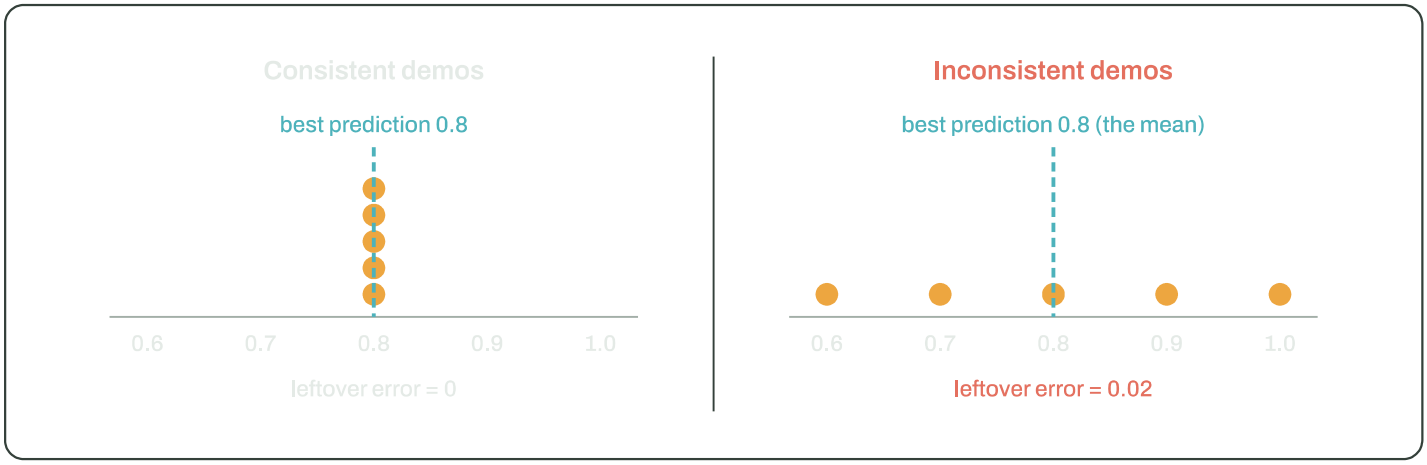


Figure 6c. The floor is the spread in your demonstrations. The dots are five demonstrations' gripper-close values for the same situation.

FLOOR = AVERAGE SQUARED DISTANCE FROM THE MEAN

$$(0.2^2 + 0.1^2 + 0^2 + 0.1^2 + 0.2^2) / 5 = (0.04 + 0.01 + 0 + 0.01 + 0.04) / 5 = 0.02$$

The flow-matching loss is not literally this formula, but it behaves the same way: the less predictable your demonstrations are from what the cameras show, the higher the floor. That is also why loss values cannot be compared across different datasets. One published SO-101 run reached about 0.018 on a single, tidy pick-and-place task; a two-instruction dataset recorded by several operators can settle higher and still produce a good policy.

WORKED CASE STUDY

"20,000 steps, the loss fell to 0.08 and then stayed flat"

This is the most common question after a first fine-tune, and a good example of tuning methodically. A flat loss after 10,000+ steps is normal (shape A). The real questions are whether the floor is higher than it needs to be, and whether the robot works anyway. Work through the checks in order: the cheap checks that catch the most problems come first.

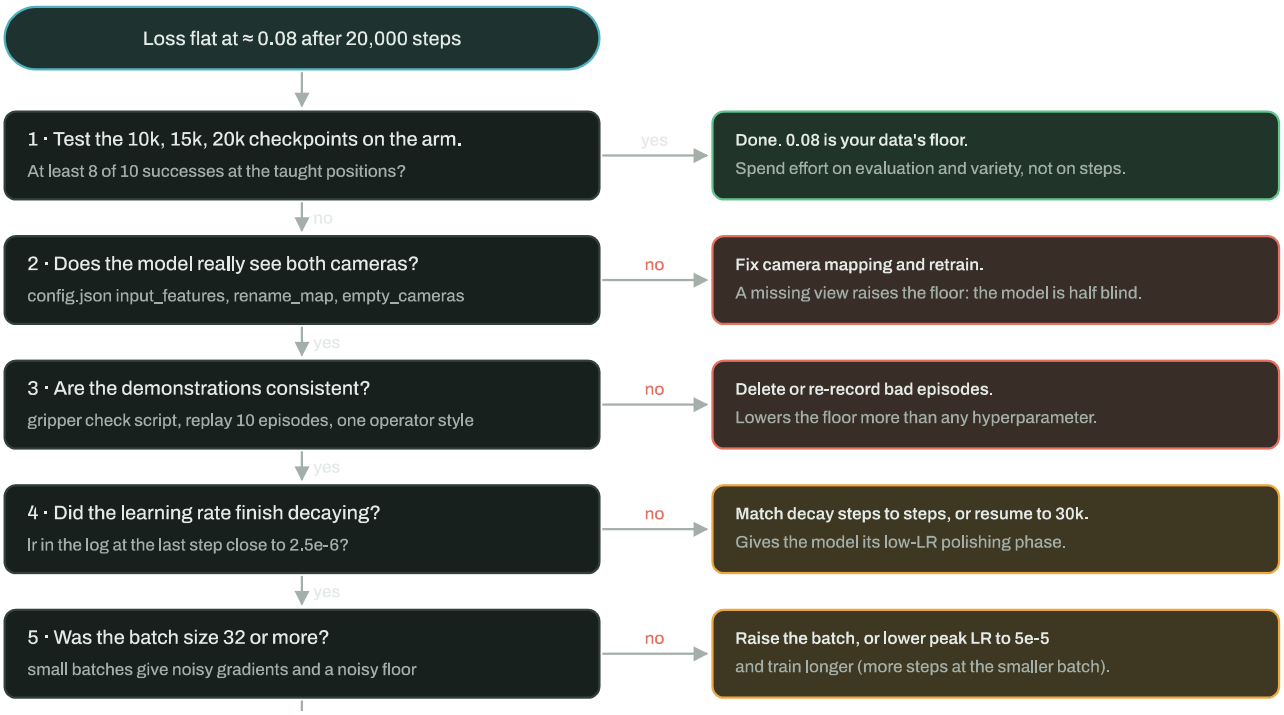




Figure 6d. Diagnosing a plateau, cheapest and most likely check first.

POSSIBLE CAUSE OF THE 0.08 FLOOR	HOW TO CHECK	ACTION
Nothing is wrong: it is the natural floor of your data	Robot succeeds on the taught positions	Stop tuning; move on to Stage 6 evaluation and to variety
A camera is missing or mapped to an empty slot	Open the checkpoint's `config.json`: `input_features` should list both real cameras; the training log warns about missing keys	Correct `--rename_map` and `--policy.empty_cameras`; retrain from `smolvla_base`
Inconsistent or failed demonstrations	Gripper-change script (Section 3.4); replay episodes; compare operators	Delete bad episodes, re-record in one style; this is the strongest lever
Wrong or duplicated task strings	`meta/tasks` lists more tasks than you meant (typos, extra spaces)	Fix the strings so each task has exactly one
Schedule cut short	Last logged `lr` is still around 2.7e-5, not 2.5e-6	`--policy.scheduler_decay_steps=20000` on a new run, or resume to 30,000 steps
Batch too small for the learning rate	You used 8–16 to fit the GPU	Lower peak LR to 5e-5 and increase steps; or train with a bigger batch on a larger GPU
Outliers in the data	`meta/stats.json`: a joint with a huge σ, or one that never moves (σ near 0)	Find and remove the episode with the bad values; recalibrate if needed
Frozen backbone limits the fit (least likely for pick-and-place)	All of the above are ruled out and you have many episodes	Try `--policy.use_peft=true` or

	`--policy.train_expert_only=false` ; watch memory and overfitting

Commands for the most likely next runs

```
# Option 1 · continue the same run to 30,000 steps (keeps optimiser and schedule state
lerobot-train \
  --config_path=outputs/train/smolvla_redblue/checkpoints/last/pretrained_model/train_
  --resume=true --steps=30000

# Option 2 · new run with a schedule that finishes, a gentler peak, and lighting augme
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/so101_redblue_bin_clean \
  --batch_size=32 --steps=30000 --save_freq=5000 \
  --policy.optimizer_lr=5e-5 \
  --policy.scheduler_warmup_steps=1000 \
  --policy.scheduler_decay_steps=30000 \
  --dataset.image_transforms.enable=true \
  --output_dir=outputs/train/smolvla_redblue_v2 --job_name=smolvla_redblue_v2 \
  --policy.device=cuda --policy.repo_id=${HF_USER}/smolvla_redblue_v2 --wandb.enable=
```

Image augmentation usually makes the training loss settle slightly *higher*, because the task got harder, while making the robot more tolerant of lighting changes. A higher loss is not always a worse policy.

4.5 A practical tuning routine

1. **Run the defaults first** with the largest batch that fits, `--save_freq=5000` and Weights & Biases on. This is your baseline.
2. **Read four numbers in the log:** `loss` (smoothed trend), `lr` (does it finish low?), `grdn` (gradient norm; repeated values near the clip of 10 mean violent updates) and `data_s` vs `updt_s` (if loading data takes longer than the update, add `--num_workers`).
3. **Test checkpoints on the robot:** 10 trials each for 10k, 15k and 20k at the taught positions. Pick by success, not by loss.

positions. Pick by success, not by loss.

4. **Fix data before settings.** Run the Section 3.4 checks; most plateaus and most failures trace back to the episodes.

5. **Change one thing per run**, keep the seed fixed, and write each run in a log like the one below.

RUN	CHANGE FROM PREVIOUS	BATCH	STEPS	FINAL LR	LOSS (SMOOTHED)	SUCCESS (10 TRIAL
A	Defaults (decay 30k)	your value	20,000	2.7e-5	0.08 (flat)	te
B	Removed flagged episodes	same	20,000			
C	Decay steps = 20,000	same	20,000	2.5e-6		
D	Peak LR 5e-5, 30k steps	same	30,000			

TRAINING LOSS IS NOT VALIDATION

LeRobot reports training loss only. A low training loss can mean the model memorised your episodes. A simple guard: keep 5–6 episodes out of training (select the rest with `--dataset.episodes`) and, after training, compare the model's predicted actions with those held-out recordings, or simply judge checkpoints on the robot at positions between the taped spots (Stage 6, test T3).

What goes wrong here, and how to fix it

CHALLENGE	WHAT YOU SEE	WHY	HOW TO OVERCOME IT
Out of memory	`CUDA out of memory`	Batch too big for the GPU	Lower `--batch_size`; keep the backbone frozen (default); close other GPU programs; or train on Colab / a cloud A100.
Feature mismatch	Error listing missing `observation.images`	Your camera names differ from the base model's	`--rename_map` and `--policy.empty_cameras` (step 2).

	`.*` keys		
Very slow steps	GPU usage low, CPU at 100%	Decoding MP4 video is the bottleneck	Increase dataloader workers (`--num_workers`); keep the dataset on a local SSD.
Training takes too long	Hours per thousand steps on a laptop	Small GPU, small batch	Start the run at the end of a lab and let it run overnight, or use cloud time. ~4 h on one A100 for 20k steps at batch 64.
Loss plateaus	Loss flat at a level such as 0.08 for thousands of steps	Usually the data's floor; sometimes a missing camera or a schedule cut short	Follow the case study in 4.4 before changing settings.
Memorising	Loss very low, robot fails at positions between the taped spots	Too few, too similar episodes, seen 30+ times each	Compare checkpoints at 10k, 15k and 20k on the arm; add episodes rather than steps.
Language forgotten	Ignores instructions it should know	Data design (Stage 3), sometimes long training on narrow data	Fix the data first; test earlier checkpoints with the instruction-swap test.

STAGE 5 OF 6 DEPLOYMENT

Deploy on the real arm

Output of this stage: the follower arm running your model in real time, safely, with smooth motion between chunks.

Pre-flight checklist

- Same `--robot.id` as in recording (same calibration file).
- Same camera **names** (`front` , `wrist`), same **positions**, same resolution and fps.
- Same **instruction string**, character for character, as in the dataset.
- Same lighting; fragile things removed from the table.

- One person's hand next to the follower's power switch for the whole run.

Steps

1. **Run the policy.** In LeRobot v0.6 the `lerobot-rollout` command runs a trained policy on the robot (older versions used `lerobot-record` with `--policy.path`).

```
source rig.env
lerobot-rollout \
  --strategy.type=base \
  --robot.type=so101_follower --robot.port=$FOLLOWER_PORT --robot.id=$FOLLOWER_I
  --robot.cameras="{ front: {type: opencv, index_or_path: $FRONT_CAM, width: 640
  --task="Put the red cube in the bin" \
  --policy.path=${HF_USER}/smolvla_redblue \
  --display_data=true
```

2. **Start slow and close.** First run with the red cube at position 3 only. Watch the first chunk with your hand on the power switch.

3. **If the motion pauses or jerks between chunks**, read the timing section below and try real-time chunking: add `--inference.type=rtc`.

4. **Change only the instruction** to "blue" and run again with both cubes on the table. This is your first quick listening check.

Why the arm can stutter: the timing maths

The model needs some time L to compute a chunk (its **latency**). While it is thinking, the clock keeps running at $\Delta t = 1/\text{fps}$ per step.

STEPS THAT PASS DURING ONE INFERENCE

$$k = \lceil L / \Delta t \rceil$$

SYNCHRONOUS: FRACTION OF TIME THE ARM STANDS STILL

$$\text{frozen} = \frac{L}{n \cdot \Delta t + L} \quad (n = \text{steps executed per chunk})$$

The fix is **asynchronous inference**: start computing the next chunk while the arm is still playing the current one. Ask for it when only g steps are left in the queue. As long as $g \geq k$, the new chunk arrives before the old one runs out, and the arm never stops. SmolVLA was designed for this; LeRobot's real-time chunking (RTC) also blends the new chunk into the old one so there is no jump at the boundary.

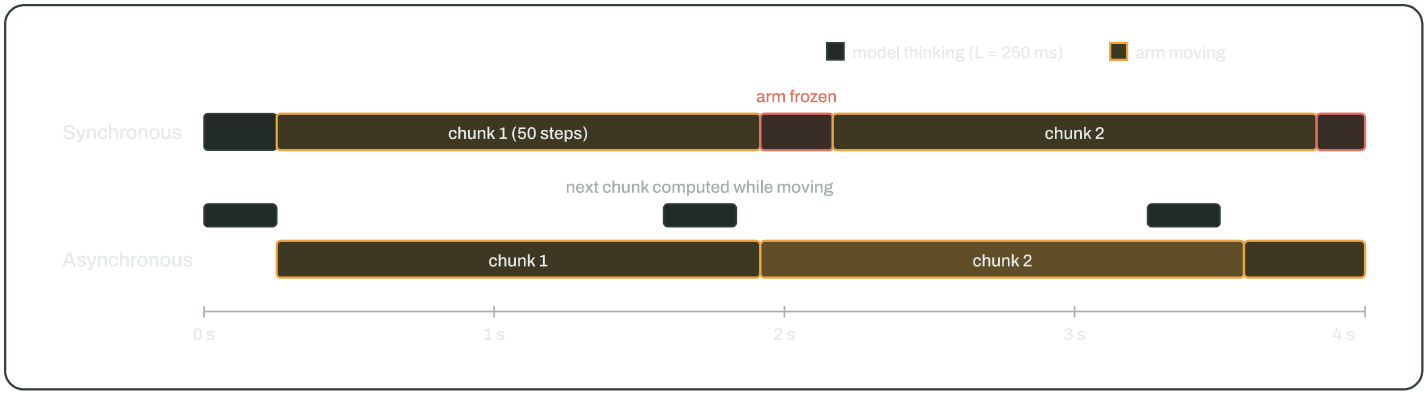


Figure 7. Same model, two schedules, drawn to scale for 30 Hz, 50-step chunks and an assumed 250 ms latency. Measure your own latency; it depends on the GPU.

CALCULATOR **Will my arm stutter?**

CONTROL RATE (HZ)	STEPS EXECUTED PER CHUNK	INFERENCE LATENCY (MS)
30	50	250

33.3 ms	1.67 s	8	13.0%
Δt per step	motion per chunk	steps that pass while thinking (k)	time frozen if synchronous

≥ 8

queue threshold g for async

Synchronously, the arm stops for 250 ms every 1.67 s (13.0% of the time). Asynchronously, request the next chunk with at least 8 steps left (add a margin, e.g. 10).

SAFETY

- Keep the follower's power switch within reach; cutting power is your emergency

stop.

- The follower config has a `max_relative_target` setting that limits how far a joint may move in one step. Set it low for first tests (check its units with `--help`).
- Clear the table of anything that is not part of the task, and keep fingers out of the gripper's path.
- Give servos a rest every 15–20 minutes of continuous running; hot servos lose accuracy.

What goes wrong here, and how to fix it

CHALLENGE	WHAT YOU SEE	WHY	HOW TO OVERCOME IT
Pauses between chunks	Arm stops briefly every 1–2 s	Synchronous inference	Asynchronous inference or `--inference.type=rtc`; a faster GPU.
Jerk at chunk boundaries	Sudden jump when a new chunk starts	New plan disagrees with the old one	RTC blending; execute fewer steps per chunk so plans overlap.
Sudden wild move at start	Arm lunges to a strange pose	Wrong calibration ID or wrong camera order	Check `rig.env`; low `max_relative_target` while testing.
Freezes near the cube	Hovers, opens and closes, never commits	Two valid options look equally good, or an unfamiliar view	More demos at that position; check the wrist camera sees the cube.
Worked in the lab, fails elsewhere	Success drops on a new table or under new light	Distribution shift: the images look new to the model	Record some episodes in the new setting and fine-tune again.

STAGE 6 OF 6 EVALUATION

Evaluate honestly

Output of this stage: a score sheet with success rates and their error bars, the result of the instruction-swap test, and a list of failures each traced to a stage.

6.1 A fixed test protocol

Decide the trials before you start, write them on a sheet, and do not change the plan when the robot is having a good or bad day.

TEST	SET-UP	TRIALS	QUESTION IT ANSWERS
T1 · In-distribution	Taped positions 1–5, both instructions, as recorded	5 × 2 × 2 = 20	Did it learn the task at all?
T2 · Instruction swap	Both cubes placed; say "red", then the same scene with "blue"	10 pairs	Does it listen? (the central failure mode)
T3 · New positions	Cubes placed between the taped marks	10	Does it generalise in space?
T4 · Distractor	Add a green cube that is never mentioned	10	Is it confused by new objects?
T5 · Lighting	Lamp moved or dimmed	10	How fragile is its vision?

Score each trial as `success` (cube in bin), `partial` (grasped but not placed) or `fail`, and write one word on *how* it failed. For T2, a pair counts as "listened" only if both runs move the named cube.

6.2 How sure can you be? The error bar

Twenty trials sound like a lot. They are not. If the robot succeeds s times in N trials:

SUCCESS RATE AND ITS 95% MARGIN

$$p = \frac{s}{N} \qquad \text{margin} \approx 1.96 \times \sqrt{\,p(1-p) \,/\, N\,}$$

WORKED EXAMPLE	p	MARGIN	PLAUSIBLE RANGE
14 of 20	70%	±20%	50% – 90%

56 of 80	70%	±10%	60% – 80%

So if checkpoint A scores 13/20 and checkpoint B 15/20, you have **not** shown that B is better. Quadrupling the trials halves the margin.

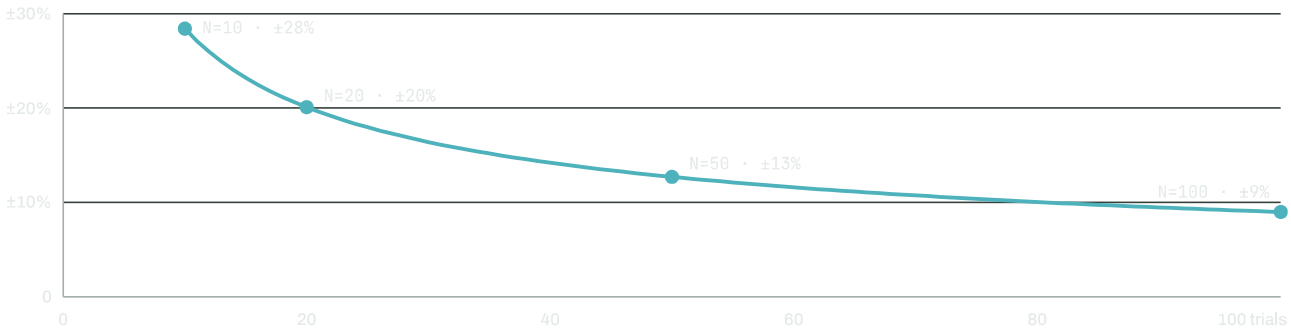


Figure 8. Error bar versus number of trials for a true success rate near 70% (computed with the formula above).

CALCULATOR **Success rate with error bar**

SUCCESSES: 14

TRIALS: 20

70%	±20%	50% – 90%	80
success rate p	95% margin	plausible range	trials needed to halve the margin

The true rate is probably between 50% and 90%. Report it as "70% ± 20% (14/20)".

The simple formula is rough when p is close to 0% or 100% or N is very small; it is good enough for comparing lab runs.

6.3 From failure to fix: trace it back

Every failure points to a stage. Most of the time the fix is better data, not more training.

FAILURE TYPE	TYPICAL SIGN	MOST LIKELY CAUSE	GO BACK TO

Language	Fails T2: picks the same cube for both instructions	Instruction did not decide the target in the data	Stage 3 · paired scenes
Perception	Reaches to where the cube *usually* is	Camera moved; cube hidden; too few positions	Stage 1 / 3
Grasp	Closes too early or too late; cube slips	Inconsistent grasps in demos; wrist camera view poor	Stage 3 · demo quality
Timing	Stutters, pauses, jerks	Latency and chunk schedule	Stage 5 · async / RTC
Distribution shift	Fails T4 or T5 only	The test scene looks unlike the training scenes	Stage 3 · add varied episodes
Memorising	Passes T1, fails T3	Too few episodes, trained long	Stage 3 / 4 · more data, compare checkpoints

PART C1 PUTTING IT INTO PRACTICE

Suggested lab plan

Four 3-hour lab sessions for a team of 3–4 learners sharing one leader–follower pair. Each session's output is the next session's input, so a lab can pause and resume without losing work.

SESSION	STAGES	TEAM ROLES	OUTPUT HANDED TO THE NEXT SESSION	ASSESSMENT EVIDENCE
Lab 1 Rig & software	1, 2	Builder, installer, tester	`rig.env` , calibrated arms, teleop working with 2 cameras, a 10-episode pilot dataset	Teleop video; photo of bench with camera reference
Lab 2 Data	3	Operator, scene-setter, logger (rotate	60-episode dataset on the Hub; recording log; training started at the end of the lab	Data-design sheet showing the "cover the instruction" test

		every block)		
Between labs	4	—	Checkpoints at 5k, 10k, 15k, 20k steps	Loss curve screenshot
Lab 3 Deploy & evaluate	5, 6	Safety officer, operator, scorer	Score sheet T1–T5 with error bars; failure log	Instruction-swap result; failure → stage table
Lab 4 (optional) Improve	3 → 6	Whole team	One targeted data fix (e.g. 20 episodes at the worst position), fine-tune again, re-evaluate; ACT on the same data as a baseline	Before/after comparison with error bars

Mapping to the unit's learning outcomes: dataset generation (Lab 2), imitation-learning policy (ACT baseline, Lab 4), VLA integration (Labs 2–3), policy evaluation and failure-mode analysis (Labs 3–4).

PART C2 QUICK REFERENCE

Troubleshooting index

The first thing to try for the problems learners hit most often.

SYMPTOM	FIRST THING TO TRY	DETAILS
Follower does not move during teleop	Re-run `lerobot-find-port` ; ports may have swapped	Stage 2
Arm jumps to an odd pose on start	Check `--robot.id` matches the calibration	Stage 2
Unrecognised command-line flag	`--help` on that command; your LeRobot version differs	Stage 2
CUDA out of memory	Halve `--batch_size`	Stage 4
Missing `observation.images.*` keys	`--rename_map` and `--policy.empty_cameras`	Stage 4

Loss is low but robot fails	Test earlier checkpoints; inspect demos for that position	Stage 4, 6
Same cube whatever the instruction	Instruction-swap test; redesign data with paired scenes	Stage 3
Arm pauses every second or two	`--inference.type=rtc`	Stage 5
Worked yesterday, not today	Compare today's camera view with the reference photo; check lighting	Stage 1

PART C3 REFERENCE

Glossary

VLA	Vision-Language-Action model: takes images and an instruction, outputs robot actions.
VLM	Vision-Language Model: takes images and text, outputs text. The "brain" inside a VLA.
Token	A list of numbers representing one small piece of input: an image patch, a word piece, or the joint state.
Action	The 6 target joint positions sent to the follower at one time step.
Action chunk	A block of the next H actions (50 for SmolVLA) predicted in one go.
State	The 6 joint positions the follower measures right now.
Episode	One recorded attempt at the task, from start pose to finish.
Teleoperation	Controlling the follower by moving the leader arm by hand.
Pre-training / fine-tuning	Pre-training: the large first training on many datasets, done by the model's authors. Fine-tuning: your short extra training on your own data.
Frozen	Part of the network whose weights are not changed during fine-tuning

Frozen	Part of the network whose weights are not changed during fine-tuning.
Flow matching	A way to generate outputs by starting from noise and following a learned velocity towards realistic data.
Normalisation	Rescaling numbers with $(a - \mu)/\sigma$ so all joints share a similar range.
Latency	Time the model takes to compute one chunk.
Asynchronous inference / RTC	Computing the next chunk while the current one plays; real-time chunking also blends the two smoothly.
LoRA	Low-Rank Adaptation: trains small add-on matrices instead of the whole network, to save memory.
Distribution shift	When test conditions (light, positions, objects) differ from the training data.
Checkpoint	A saved copy of the model's weights at one point in training.

Intentionally left out

To keep this lesson at beginner level, the following are deliberately not covered here:

- **Full transformer and attention maths** (queries, keys, values, multi-head attention). Only the softmax weighting is shown.
- **π0 / π0.5 internals**: the FAST action tokenizer, knowledge insulation and co-training with web data.
- **GR00T N1.x fine-tuning workflow** (its modality configuration and separate training scripts). Mentioned for comparison only.
- **World action models and RL fine-tuning of VLAs**. These belong to the next part of the learning outcome on VLA and world-model integration.
- **The Isaac Lab simulated-data pipeline in detail**. It is covered in the imitation-learning lab; here it appears only as an optional data source.
- **Measured GPU memory and latency** for the lab laptops. The figures here are rules of thumb and an assumed 250 ms latency; measure on your own hardware.
- **Bimanual or mobile set-ups** and more than two cameras.

Commands were checked against the LeRobot documentation for v0.6.x in November

2026. LeRobot changes quickly, and command names and flags differ between releases; when this page and `--help` disagree, trust `--help`. Numbers marked illustrative are for teaching, not measurement.

Sources

- LeRobot docs · SmolVLA
- LeRobot docs · SO-101
- LeRobot docs · Imitation learning on real robots
- LeRobot release notes (v0.5–v0.6)
- Arm Learning Path · Fine-tune SmolVLA for SO-101
- NVIDIA · Post-training GR00T N1.5 for SO-101
- AY-Robots · Training π0.5 on SO-101
- Shukor et al. · SmolVLA paper (2025)
- Black et al. · π0 paper (2024)
- Lipman et al. · Flow Matching (2022)